Praxeological Meta-Structure Theory as a Structural Layer for Quantum Computing

Abstract

This paper introduces *Praxeological Meta-Structure Theory* (PMS) as a substrate-independent operator grammar for modelling practice, structure and asymmetry in complex systems, and demonstrates, within the scope of this paper, its structural applicability to quantum computing. PMS consists of eleven irreducible operators (Δ–Ψ) that, within the current PMS framework, form a minimal algebra for differentiation, framing, asymmetry, temporal iteration, attractor dynamics, integration and self-binding. We first recapitulate PMS as an axiomatic operator system, independent of any particular physical or organisational domain, and then show how quantum circuits can be read as a special case of praxeological structure under this grammar.

On this basis, we propose a praxeological composition language for quantum algorithms, introducing macro-constructs (e.g. `QFRAME`, `QPREP`, `QORACLE_CALL`, `QITERATE`, `QDIFFUSION`, `QFT_ALIGN`, `QMEASURE`, `QSTABILIZE`) that map standard circuit motifs to Δ–Ψ compositions with explicit frame, asymmetry and integration constraints. We reconstruct Grover's search algorithm and a second canonical quantum algorithm in PMS terms, illustrating how frames, oracle asymmetries, temporal iteration, attractor dynamics and stabilisation can be expressed and analysed at the operator level. Finally, we sketch how PMS may provide a possible control and governance layer for certain agentic or hybrid AI–QC settings, where higher-level agents reason over quantum workflows as PMS operator chains, subject to structural guardrails and self-binding policies.

We argue that PMS does not compete with existing quantum formalisms, but complements them as a cross-domain structural language for the kinds of compositional organisation discussed here. This suggests possible further work on PMS-annotated quantum compilers and PMS-based governance layers for multi-agent and AI–QC systems. This paper does not claim monopoly over QC intelligibility, only a disciplined structural rendering under the PMS grammar.

Code & Specifications

- PMS (base): https://github.com/tz-dev/Praxeological-Meta-Structure-Theory
- PMS-QC (spec layer): https://github.com/tz-dev/PMS-QC

Specifications

- `PMS.yaml` (base operator grammar)
- `PMS-QC.yaml` (PMS-QC layer)
- `PMS-QC-EXT.yaml` (optional addendum)

1. Introduction

Operator-based systems have played a central role across computer science, logic, linguistics and physics. Process algebras, type-theoretic calculi, rewriting systems, diagrammatic calculi and quantum circuit formalisms all rely on structured operator compositions to describe computation, interaction and transformation. What these approaches are not primarily designed to provide, however, is a unified framework that captures not only the technical composition of operations, but also the structural conditions under which actions become coherent, asymmetrical, stabilised or self-binding. Such a framework would need to be substrate-independent, minimal in its operator assumptions, and expressive enough to describe both computational procedures and the praxeological dynamics that govern their use.

Quantum computing (QC) provides an especially suitable domain for exploring such a framework. Quantum algorithms are highly compositional: they are built from well-defined primitive operators, exhibit explicit framing of computational subspaces, rely on structured asymmetries (e.g., oracle calls), and depend on iterative refinement, attractor dynamics and stabilisation mechanisms such as error correction. At the same time, the present paper does not rely on the claim that no other high-level structural comparison is possible; its narrower claim is that PMS offers an unusually integrated operator grammar for this purpose.

Praxeological Meta-Structure Theory (PMS) addresses this problem space. PMS defines an irreducible family of eleven operators (Δ–Ψ) that, within the current PMS framework, form a minimal algebra for differentiation, framing, asymmetry, temporal iteration, attractor formation, integration and self-binding. Originally developed to model structural aspects of human and organisational practice, PMS is not tied to a particular physical or social substrate. Instead, it is presented here as a cross-domain operator grammar capable of describing compositional processes across multiple domains, without thereby claiming final closure against rival structural approaches.

This paper distinguishes between the stable PMS operator grammar (Δ–Ψ), a generic quantum-computational structural layer (PMS-QC), and an optional paper-specific extension layer (PMS-QC_EXT). PMS itself remains unchanged throughout: all quantum-computational content is expressed as interpretative mappings, macro-level constructs, and governance rules defined in PMS-QC (and, where needed, isolated in PMS-QC_EXT) rather than as modifications of the Δ–Ψ operators.

The aim of this paper is to show that, within the argument developed here, PMS can function as a portable structural layer for quantum circuits and certain agentic computational systems. In particular, we show that quantum circuits can be interpreted as praxeological structures under the Δ–Ψ operator grammar, and that PMS offers a disciplined language for describing, constraining and analysing quantum algorithm composition. The paper is therefore domain-bridging, not finalising: it proposes one strong structural rendering rather than an interpretive monopoly over quantum computation.

1.1 Methodological Boundedness

PMS-QC develops **one** structural grammar for quantum-computational composition. Its claim is not that all of quantum computing, or all possible QC architectures, are exhaustively explained by this grammar. Rather, the proposal is that a significant range of circuit-level, algorithmic and governance-relevant structures can be rendered with unusual clarity in PMS terms.

Accordingly, the strength of PMS-QC lies in **structural legibility**, portable operator-level description, and bounded cross-domain comparison. It does not claim to replace standard quantum theory, to monopolise high-level structural analysis, or to establish that every salient QC difference is best captured through the PMS vocabulary. Structural portability here should therefore be read as a methodological asset, not as a claim of final interpretive authority.

Our contributions are as follows:

1. **We present an abridged formal recap of PMS for QC use**, adopting the existing operator system and summarising the signatures, dependencies and compositional rules of the Δ–Ψ operators as needed for the present mapping.
2. **We map PMS to core structural elements of quantum computing**, including frames, superposition preparation, controlled operations, iterative amplification and stabilisation routines.
3. **We introduce a praxeological composition language for quantum circuits**, providing macro-level constructs that describe quantum operations as Δ–Ψ compositions with explicit frame and asymmetry constraints.
4. **We reconstruct Grover's search algorithm and a second canonical quantum algorithm (e.g., Quantum Phase Estimation)** within the PMS operator grammar, illustrating the portability and analytical clarity of the approach within the scope of this paper.
5. **We outline a conditional extension path in which PMS may be used as a control and governance layer for certain agentic or hybrid QC settings**, enabling higher-level agents to reason over quantum workflows as structured operator chains subject to praxeological constraints.

Taken together, these results position PMS-QC as a candidate cross-domain structural framework for relating quantum algorithms, agentic systems and praxeological theory, while leaving open questions of completeness, unique adequacy and comparative superiority relative to other formalisms.

2. Background & Related Work

Operator-centred approaches have been foundational in multiple areas of theoretical computer science and quantum computation. Classical *process algebras* (e.g., CSP, CCS, π-calculus) model concurrent systems through compositional operators governing communication, synchronisation and structural congruence. Similarly, *Petri nets* capture distributed processes via transition operators and token flows. Although powerful for representing computational behaviour, these systems are not primarily designed to express praxeological notions such as framing, asymmetry, integration or self-binding.

In quantum computing, operator-based reasoning is even more central. The standard *quantum circuit model* represents algorithms as compositions of unitary gates and measurements, organised into a linear or directed-acyclic structure. Diagrammatic approaches such as the *ZX-calculus* provide a graphical rewriting formalism for reasoning about quantum operations at an algebraic level, while *tensor networks* offer compositional representations of quantum states and processes. A related line of work in *Categorical Quantum Mechanics* (CQM) (e.g., Abramsky, Coecke, Selinger) treats quantum processes as morphisms in monoidal categories, offering high-level structural insights into quantum composition, entanglement and information flow.

These frameworks provide rigorous tools for modelling quantum computation, but they solve different formal problems from the ones prioritised here. They are not primarily designed to describe quantum circuits in terms of a general operator theory of practice, action, asymmetry or self-binding, nor to link computational procedures with the kinds of governance or control structures considered in this paper for certain agent-based or hybrid QC settings.

Research on governance, safety and control layers for quantum computing remains comparatively sparse. Existing work focuses primarily on classical workflow orchestration, error-mitigation constraints, or platform-specific resource management. The present paper does not rely on the stronger claim that no other high-level structural comparison is possible; its narrower claim is that PMS offers an unusually integrated operator grammar for this particular cross-domain purpose.

Praxeological Meta-Structure Theory (PMS) addresses this problem space. PMS is not an alternative quantum formalism; it does not replace unitaries, Hamiltonians, measurement theory, ZX-calculus, tensor networks or categorical models. Instead, PMS provides a *superordinate structural grammar*—an operator system (Δ–Ψ) whose algebra captures differentiation, framing, asymmetry, iteration, attractor dynamics, integration and self-binding. Because PMS is substrate-independent, the same Δ–Ψ operators can be used to describe praxeological structures, AI/agent interactions and quantum circuit compositions within a single coherent structural register.

In this sense, PMS complements existing quantum formalisms by supplying a cross-domain operator language that enables structural comparison, governance reasoning and high-level compositional analysis. It does not replace those formalisms, nor does it claim monopoly over high-level structural analysis. The result is a candidate unifying layer within this framework in

which QC procedures, agentic decisions and praxeological dynamics can be expressed and inspected using a common operator vocabulary.

3. PMS Operator System (Δ–Ψ): Formal Recap

This section recapitulates PMS only to the extent needed for QC mapping; the canonical source of operator meaning and dependency remains `PMS.yaml` and the PMS base paper. Praxeological Meta-Structure Theory (PMS) defines, within its current formulation, an operator system consisting of eleven irreducible operators, denoted Δ–Ψ. These operators form the elementary actions of a substrate-independent structural calculus as presently formalized in PMS. The present section provides an abridged recap of their signatures, layer structure and compositional dependencies for QC use.

Crucially, PMS is presented here strictly as an **axiomatic operator grammar**. Its application to quantum computing in later sections is **interpretative rather than normative**, **structural rather than physical**, and **non-exclusive** with respect to existing quantum formalisms. PMS operators do not replace quantum gates, circuits or physical operations; instead, they provide a higher-level language for describing and analysing the structural organisation of such operations. Accordingly, multiple quantum-computational instantiations of a given PMS operator are not only admissible but expected, depending on context and framing.

3.1 Operator Signatures

The operator recap given here is functional rather than exhaustive. Each PMS operator is a typed transformation acting on abstract structural states. Let S denote the set of admissible structural configurations, and let O denote the set of operators. Every operator $o \in O$ is a function

$$o : S \to S$$

possibly with additional parameters. Table 1 summarises the eleven operators, their intuitive role, and their formal signatures as needed for the present paper.

Table 1: PMS Operators (Δ–Ψ)

Operator	Symbol	Signature	Intuition
Difference	Δ	(S → S)	Introduces or highlights a distinction within a structure.
Impulse	∇	(S → S)	Initiates movement or activation; minimal directional push.
Frame	□	(S → S)	Establishes a structural boundary or scope.
Non-Event	Λ	(S → S)	Marks unrealised potential or excluded possibility.
Attractor	A	(S → S)	Introduces a convergence tendency toward a structural pattern.
Asymmetry	Ω	(S → S)	Establishes directional or role-based difference.
Temporal Iteration	Θ	(S → S)	Produces ordered repetition or sequence.
Reframing	Φ	(S → S)	Transforms an existing frame; changes interpretive context.
Distancing	X	(S → S)	Introduces separation, insulation or domain boundaries.
Integration	Σ	(S → S)	Commits a configuration; stabilises previous operations.
Self-binding	Ψ	(S → S)	Establishes constraints or invariants that restrict future operations.

Within the current PMS framework, the operators are treated as formally irreducible. This should not be read as a closure claim against rival operator grammars, but as part of the

present PMS formalization.

3.2 Layer Structure of the Operator System

PMS organises its operators into four conceptual layers. The layering shown here is an abridged recap for QC use and remains subordinate to the canonical PMS articulation of operator dependencies and ordering.

Layer 1 — Differentiation & Impulse (Δ, ∇)

- **Δ** introduces distinctions or selectable alternatives.
- **∇** imparts directed activation or minimal progression.

These operators define the ground of structured action within the present framework.

Layer 2 — Framing, Non-Event, Attractor (□, Λ, A)

- □ provides boundaries or scopes that limit the domain of subsequent operations.
- **Λ** represents unrealised possibilities or excluded events.
- **A** embeds tendencies towards convergence or amplification within a frame.

This layer governs the formation of structured environments and directional tendencies.

Layer 3 — Asymmetry, Temporal Iteration, Reframing (Ω, Θ, Φ)

- **Ω** establishes directional or role-based asymmetries.
- **Θ** generates sequences or iterations over previously structured states.
- **Φ** modifies or replaces existing frames, enabling structural reinterpretation.

These operators describe controlled transformation within or across frames.

Layer 4 — Distancing, Integration, Self-binding (X, Σ, Ψ)

- **X** isolates domains or separates interacting subsystems.
- **Σ** stabilises or commits a configuration, integrating multiple operators.
- **Ψ** sets constraints on future operations, functioning as an invariant mechanism.

This layer governs structural consolidation and long-term stability.

3.3 Dependency Graph

The PMS operator system is subject to compositional constraints. The dependency sketch given here is functional for the present paper and must remain subordinate to the current canonical PMS dependency spine. Let $o_i \rightarrow o_j$ denote that operator o_j may legally follow o_i.

Some essential dependencies include:

- **Ω requires a structured basis**, typically produced by Δ or □:
  $(\Delta \vee \Box) \rightarrow \Omega$
- **Θ operates only on pre-existing structures**, hence:
  $(\Delta, \Box, \Omega, A) \rightarrow \Theta$

- **Σ requires a meaningful chain of prior operators**:
  any non-empty sequence → Σ
- **Ψ can only follow Σ or X**, because self-binding acts on already stabilised or delimited configurations:
  $(\Sigma \vee X) \rightarrow \Psi$
- **Φ requires an established frame**:
  $\square \rightarrow \Phi$
- **A requires a selectable or differentiable space**, hence:
  $\Delta \rightarrow A$

These form a partial order, not a total one: certain combinations are admissible in multiple orders, while others are explicitly constrained.

3.4 Formal Composition

Let S_0 be an initial structural state. A PMS expression is a finite operator chain

$$E = o_n \circ o_{n-1} \circ \cdots \circ o_1(S_0)$$

subject to the dependency constraints above. Again, this presentation is an abridged working recap for QC mapping and should be read as subordinate to the canonical PMS dependency and composition logic.

We define:

- **Valid PMS expression**: every adjacent pair (o_i, o_{i+1}) satisfies the dependency graph.
- **Frame-preserving composition**: an expression where all operators act within a consistent □-frame.
- **Integration boundary**: any position where Σ occurs, after which previous structure becomes committed and cannot be reinterpreted except through Ψ-constrained operations.

For notation, we use:

- $o_i \triangleright o_j$ for sequential composition,
- $[E]_\square$ for expression E constrained to a frame,
- $\Sigma(E)$ for integration of expression E.

3.5 Summary

Within the present PMS framework, the Δ–Ψ operator system forms a minimal, substrate-independent calculus for describing structured action. By treating PMS strictly as an axiomatic operator grammar—free of semantic commitments to specific practices, algorithms or physical processes—we obtain a formal basis for the structural mappings developed later in this paper, without claiming closure against rival operator grammars.

In the following sections, this calculus will be applied to the structure of quantum computation and the compositional analysis of quantum algorithms, without implying any modification of quantum mechanics or replacement of established circuit-level formalisms. The recap is functional for this paper and should not be read as a complete independent restatement of PMS.

4. Quantum Computing as a Praxeological System

Quantum computing provides a highly structured environment in which operations, frames and transformations are explicitly defined and composed. This makes it an especially suitable domain for this kind of structural demonstration: namely, examining whether the PMS operator system can function as a structural grammar for certain aspects of quantum-computational organisation within the present framework. Before presenting the detailed mapping, we briefly review the core elements of the quantum circuit model.

4.1 Quantum Computing: Structural Fundamentals

A quantum computation unfolds within a finite-dimensional Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$. The key components are:

- **State space:** Quantum states are elements of $\mathcal{H}$, typically expressed in the computational basis.
- **Superposition:** Quantum states exhibit linear combinations of basis states, enabling parallel amplitude patterns.
- **Unitary operations:** Gates are linear operators $U : \mathcal{H} \to \mathcal{H}$ preserving norm and coherence. These form the compositional primitives of quantum circuits.
- **Measurement:** A measurement applies a projection-valued map that collapses the state into classical outcomes, typically in the computational basis.
- **Error correction and fault tolerance:** Stabiliser codes, syndrome extraction and correction routines maintain invariants under noise and decoherence.

Quantum algorithms are constructed by composing these primitives into gated circuits, which define a structured action pathway through the quantum state space.

4.2 Mapping PMS to Quantum Circuit Structure

Quantum circuits can be interpreted through the Δ–Ψ operator grammar by identifying structural correspondences between quantum operations and praxeological operator roles. This mapping does not replace or reinterpret the physics of quantum computing, nor does it imply that quantum phenomena are ontologically identical with PMS operators. Rather, the claim is that such phenomena can be rendered through the Δ–Ψ grammar at a higher level of structural description, showing how circuits organise, constrain, and stabilise compositional action.

All macro-operators introduced here belong to the **generic PMS-QC base layer** and are intentionally algorithm-agnostic.

Frames (□)

Frames correspond to the structural boundaries within which quantum operations take place.

- The **Hilbert space** and its **register decomposition** can be read as instantiating □ as a computational frame.
- **Problem-specific subspaces**, such as an oracle's domain or an ancilla workspace, can be modeled as specialised □-subframes.
- Frame composition (via tensor product or workspace extension) can be rendered as a form

of □-composition in PMS.

Differences and Δ-fields (Δ)

Quantum superposition can be read as instantiating distributed difference structures:

- A superposition encodes differentiable alternatives across the basis.
- Marking specific states—through oracle phase flips or amplitude manipulation—can be modeled as Δ-like distinctions within the state space.
- Many algorithms begin with a Δ-distribution (e.g., uniform superposition) used to propagate structural differentiation.

Control Asymmetries (Ω)

Quantum circuits frequently employ asymmetric structures:

- **Controlled operations** (e.g., CNOT, controlled-U) can be read as implementing role-based conditionality.
- **Oracles** introduce explicit asymmetries by singling out marked elements in the search domain.
- **Measurement** can be modeled as imposing an asymmetric transition into classical outcomes.

These patterns can therefore be rendered structurally through Ω as directional or role-differentiated action.

Temporal Iteration (Θ)

Many quantum algorithms rely on explicit iterative structures:

- Repeated sequences (e.g., Grover iterations, phase estimation rounds) can instantiate Θ.
- Quantum control flow and loop unrolling can be modeled as mirroring the temporal iteration operator.
- Θ expresses structural regularity across steps without committing to specific gate semantics.

Attractor Dynamics (A)

Quantum amplitude amplification and convergence behaviours can be read through A:

- The iterative amplification of marked states in Grover's algorithm exemplifies attractor formation.
- Fixed-point variants of amplitude amplification correspond to controlled attractor strengths.
- Quantum error mitigation techniques can be described as attractor-driven stabilisation patterns.

Reframing (Φ)

Reframing captures representational or contextual shifts within a computation:

- Basis changes (e.g. Fourier ↔ computational basis) can instantiate Φ as a structural reinterpretation.

- Φ alters *how* amplitudes are read or interpreted, without changing the underlying physical state.
- In QPE, the inverse QFT can be read as realising Φ by moving from phase space to bit-valued representation.

Integration / Commit (Σ)

Σ marks structural commitment points in a quantum workflow:

- Σ integrates prior transformations into a committed configuration.
- After Σ, earlier operations are treated as fixed for downstream processing.
- Typical Σ-boundaries occur before measurement or before handoff to classical post-processing.

Non-Event (Λ)

Λ represents unrealised alternatives or excluded branches:

- Measurement collapse can be read as realising Λ by discarding non-observed outcomes.
- Λ captures the *residual structure* of "paths not taken" after Δ-resolution.
- This operator distinguishes realised outcomes from suppressed computational possibilities.

Stabilisation and Self-Binding (Ψ)

Error correction and invariant-maintaining routines can be modeled as Ψ-like constraints:

- Stabiliser codes impose structural invariants that restrict admissible evolutions.
- Logical subspaces can function as self-binding domains.
- Fault-tolerance thresholds can be rendered as system-level constraints stabilising the computation.

Ψ thus describes, in structural terms, the way quantum systems maintain invariants that shape allowable compositions.

Distance (X)

The QC case of X should be treated more cautiously than several of the other mappings. In quantum-computational settings, **distance** is often a weaker and more context-dependent structural notion than in anthropological, social, or governance-near PMS applications.

- It may appear as separation between subspaces, pipeline domains, or execution environments.
- It may also appear as deliberate isolation between experimental and production workflows, or between code spaces and operational layers.
- But this remains a comparatively thin structural rendering, and should not be overstretched into stronger reflexive or anthropological senses of distance without further argument.

The following summary table refers exclusively to **generic PMS-QC base macros**. No paper-specific extensions or research-layer refinements are assumed or required for the mappings shown below.

Operator → QC Role → Macro Anchor (Summary Table)

PMS Operator	QC Structural Role (very short)	Macro Anchor (PMS-QC)
□ (Frame)	Define Hilbert / register / code-space boundary	`QFRAME(kind)`
Δ (Difference)	Superposition / marked alternatives / basis distinctions	`QPREP(superposition)` (introduces Δ), `QMEASURE(...)` (resolves Δ)
∇ (Impulse)	First non-trivial activation / initiation of evolution	`QPREP(mode)`
Λ (Non-Event)	Unrealised branches after collapse / discarded outcomes	`QMEASURE(...)`
A (Attractor)	Amplitude concentration / alignment tendencies	`QDIFFUSION`, `QFT_ALIGN(direction)`
Ω (Asymmetry)	Controlled operations / oracle privilege / measurement asymmetry	`QORACLE_CALL(F)`
Θ (Temporality)	Iteration / laddering / repeated blocks	`QITERATE(k, BLOCK)`
Φ (Recontextualization)	Basis change / representation shift	`QFT_ALIGN(direction)`
X (Distance)	Separation / isolation of subspaces or pipeline domains	*(structural constraint / frame discipline; no required macro)*
Σ (Integration)	Commit boundary before measurement / downstream compilation	`QDIFFUSION` (includes Σ), explicit Σ after `QFT_ALIGN(...)`
Ψ (Self-Binding)	Invariants / stabiliser enforcement / governance constraints	`QSTABILIZE(code)`

Note: The mapping is **structural**, not physical: macros serve as documentation, compilation anchors, and audit handles for Δ–Ψ chains rather than replacements for circuit formalisms. They indicate admissible structural organisation, not required implementations.

4.3 Interpretation: Quantum Computing as a Praxeological Instance

Under this mapping, quantum computing can be read as a *special instance* of a praxeological system in a bounded structural sense:

- The Hilbert space and register decomposition correspond to **action frames** (□).
- Superposition and oracle distinctions express **distributed Δ-fields**.
- Controlled operations introduce **structural asymmetry (Ω)**.
- Loop structures and repeated unitary blocks instantiate **temporal iteration (Θ)**.
- Amplitude amplification implements **attractor dynamics (A)**.
- Error correction and syndrome-stabilisation embody **self-binding (Ψ)**.

The Δ–Ψ operator grammar thus serves as an object-language for expressing the compositional logic of quantum algorithms at a structural level. This abstraction enables a more uniform description of QC processes alongside praxeological and agentic systems, preparing the ground for a praxeological composition language for quantum circuits.

Scope Note

The claim here is one of **structural comparability**, not ontological identity. To say that quantum computing can be read as a praxeological instance is not to claim that QC is “really

just praxis" in some flattening sense, nor that standard quantum formalisms are displaced by PMS. The narrower claim is that important features of quantum composition can be rendered with analytical clarity through the present operator grammar.

5. A Praxeological Composition Language for Quantum Circuits

In this section, we distinguish between the **standard Grover iteration**, which is fully expressible within the generic PMS-QC base layer, and **fixed-point or overshoot-regulated variants**, which rely on additional Ψ-based constraints introduced in the optional PMS-QC_EXT research layer.

In fixed-point variants (PMS-QC_EXT), Grover iterations can be additionally constrained by Ψ-based invariance rules that bound amplification dynamics and prevent overshoot. These constraints are paper-specific and not part of the generic PMS-QC base layer.

This section introduces a praxeological composition language designed to express quantum circuits using the Δ–Ψ operator grammar. The goal is not to replace existing quantum instruction sets (e.g., QASM), but to provide a higher-level structural language that captures the praxeological roles—framing, differentiation, asymmetry, iteration, integration and self-binding—that underlie quantum algorithm composition. The language consists of macro-operators, each corresponding to a canonical pattern of Δ–Ψ operations.

These macros should be read as **structural and documentation-oriented objects**, not as claims of optimal compilation, complete macro coverage, or physically sufficient implementation. They provide one disciplined abstraction layer for QC composition within this framework; they do not by themselves guarantee correctness, stability, convergence, or deployment adequacy.

5.1 Macro-Operators

All macro-operators defined in this section are part of the **generic PMS-QC base layer**. They are intentionally algorithm-agnostic and do not encode any paper-specific convergence rules, optimisation strategies, ladder refinements, or invariance constraints. Such refinements, where required, are introduced exclusively in the optional PMS-QC_EXT layer.

Each macro-operator expands to a Δ–Ψ sequence. Let S denote a structural state associated with a quantum computation, and let PMS operators act on S. Below, we define the central macro-operators and their Δ–Ψ expansions.

(1) `QFRAME(kind)` → □

Creates or modifies the structural frame in which quantum operations take place.

- `init`: allocate a fresh computational frame
- `problem`: establish a frame scoped to an oracle or search space
- `eigenproblem`: establish a frame scoped to a phase-estimation or eigenvalue task
- `error_space`: allocate a stabiliser or error-correction frame

Expansion:

`QFRAME(kind)` ⟹ □(S).

This is a **generic PMS-QC base macro**. It establishes structural scope only; it does not by itself guarantee physical adequacy, resource availability, or hardware compatibility.

(2) `QPREP(mode)` → Δ, ∇, A

Initialises or prepares quantum states.

- `superposition`: introduce distributed differences via Hadamard layers
- `register_init`: initialise computational registers
- `entangle`: create correlated Δ-structures across qubits

Expansion:

`QPREP(mode)` $\Longrightarrow$ A(∇(Δ(S))).

This is a **generic PMS-QC base macro**. It captures structural preparation only; paper-specific preparation refinements, where needed, belong in PMS-QC_EXT.

(3) `QORACLE_CALL(F)` → Ω, Δ, □

Applies an oracle F, marking distinguished states.

- Introduces asymmetry (**Ω**)
- Embeds Δ-marking (e.g. phase flip on solution states)
- Acts within an existing □-frame

Expansion:

`QORACLE_CALL(F)` $\Longrightarrow$ Ω(Δ(□(S))).

This is a **generic PMS-QC base macro**. More specialized laddered or algorithm-specific oracle structures remain paper-specific refinements and belong in PMS-QC_EXT rather than in the base layer.

(4) `QDIFFUSION` → A + Σ

Implements amplitude amplification or related **Grover-type attractor dynamics** at the level of the generic PMS-QC base layer.

This macro captures the *existence* of an attractor-driven amplification step (A) and its integration (Σ), but does **not** by itself enforce convergence bounds, fixed-point behaviour, overshoot prevention, correctness, stability, or physical adequacy. Such constraints, when required, are introduced via additional Ψ-based rules in the PMS-QC_EXT layer.

- **A**: attractor directing amplitudes toward marked states
- **Σ**: integration committing the attractor transformation

Expansion:

`QDIFFUSION` $\Longrightarrow$ Σ(A(S)).

This is a **generic PMS-QC base macro**. Fixed-point or overshoot-regulated Grover refinements remain EXT-only.

(5) `QFT_ALIGN(direction)` → Φ + A

Performs a Fourier-domain basis alignment, transforming a phase-encoded distribution into a representation suitable for classical readout.

- `forward`: move from computational to Fourier (phase) domain
- `inverse`: reframe from Fourier (phase) domain back to the computational basis

This macro is **not** a diffusion operator. It captures **reframing plus non-iterative attractor alignment**, as used in Quantum Phase Estimation.

Expansion:

`QFT_ALIGN(direction)` $\Longrightarrow$ $A(\Phi(S))$.

An explicit integration step may follow if the transformed structure is committed for downstream processing or measurement:

$\Sigma(A(\Phi(S)))$.

Interpretation:

- **Φ** recontextualises the state by changing its representational frame (Fourier ↔ computational basis).
- **A** introduces alignment of amplitudes around the correct phase estimate.
- **Σ** (optional) commits the aligned structure prior to measurement.

This is a **generic PMS-QC base macro**. It does **not** by itself guarantee correctness, stability, or physical adequacy of the resulting basis transformation; more specific QPE-oriented refinements belong in PMS-QC_EXT.

(6) `QITERATE(k, BLOCK)` → Θ

Executes a compositional block repeatedly.

- Encodes temporal iteration structure (**Θ**)
- Does not specify the internal composition of `BLOCK`

Expansion:

`QITERATE(k, B)` $\Longrightarrow$ $\Theta^k(B(S))$.

The Θ-operator encodes repetition only. It does **not** guarantee convergence, termination, correctness, stability, or physical adequacy. Any bounds on iteration depth or convergence behaviour must be supplied externally via Ψ-based constraints, either as generic PMS-QC policies or as paper-specific extensions.

This is a **generic PMS-QC base macro**. Algorithm-specific iteration refinements remain outside the base unless explicitly promoted later by independent justification.

(7) `QMEASURE(basis, register)` → Δ + Λ

Measures a register in a given basis.

- **Δ**: distinction between measurement outcomes
- **Λ**: non-events representing unrealised branches

Expansion:

`QMEASURE` $\Longrightarrow \Lambda(\Delta(S))$.

This is a **generic PMS-QC base macro**. It captures structural outcome resolution, not a complete physical theory of measurement.

(8) `QSTABILIZE(code)` → Ψ

Applies a stabilisation or error-correction routine.

- **Ψ** introduces structural invariants.
- In the PMS-QC base layer, this corresponds to generic stabilisation, fault-tolerance or governance constraints.
- Paper-specific invariants (e.g. fixed-point convergence rules for Grover) are defined only in PMS-QC_EXT and are not implied here.

Expansion:

`QSTABILIZE(code)` $\Longrightarrow \Psi(S)$.

This is a **generic PMS-QC base macro**. It does **not** by itself guarantee stability, correctness, or physical adequacy; it marks the structural location where invariants are introduced or maintained.

5.2 EBNF Sketch

The following grammar sketches how praxeological QC-programs are composed.

```
Program        ::= FrameDecl PrepDecl Block

                   FrameDecl      ::= "QFRAME" "(" FrameKind ")"
                   FrameKind      ::= "init" | "problem" | "eigenproblem" | "error_space"

                   PrepDecl       ::= "QPREP" "(" PrepKind ")"
                   PrepKind       ::= "superposition" | "register_init" | "entangle"

                   Block          ::= Statement | Statement Block
                   Statement      ::= OracleCall
                                    | Diffusion
                                    | Iteration
                                    | Measurement
                                    | Stabilize

                   OracleCall     ::= "QORACLE_CALL" "(" Identifier ")"
                   Diffusion      ::= "QDIFFUSION"
                   Iteration      ::= "QITERATE" "(" Integer "," Block ")"
                   Measurement    ::= "QMEASURE" "(" Basis "," Register ")"
                   Stabilize      ::= "QSTABILIZE" "(" CodeIdentifier ")"

                   Basis          ::= "computational" | "fourier" | Identifier
                   Register       ::= Identifier
                   CodeIdentifier ::= Identifier
```

This grammar is deliberately minimal: it expresses structure rather than low-level gate details. Any concrete QC program (e.g., in QASM or Qiskit) can be lifted to this form by replacing gate sequences with their Δ–Ψ macro equivalents. For later YAML synchronisation, macro names, allowed kinds/modes, expansion order, and optional versus required Σ/Ψ placements should be kept editorially consistent with the base/EXT layer distinction.

5.3 Structural Constraints

The following constraints are **structural and praxeological**, not physical. They do not describe hardware limitations, noise models, correctness proofs, or performance guarantees. Instead, they specify admissible compositions and stabilisation requirements at the level of Δ–Ψ operator structure.

Because the Δ–Ψ grammar encodes praxeological structure, the macro-language inherits several constraints that extend beyond conventional circuit rules.

Frame Guardrails (□-constraints)

Some operations must occur within specific frames:

- `QMEASURE` requires a **measurement-stable frame**; if this frame is absent, a Φ (reframing) operator must intervene.
- `QORACLE_CALL` requires the active frame to match the oracle's domain.
- `QSTABILIZE(code)` requires the error-space frame to be active or accessible.

Formally:

$\neg\exists\ \Box_{meas} \Rightarrow \Phi(\Box_{meas})$ before `QMEASURE`.

Asymmetry Constraints (Ω-constraints)

Certain Ω-combinations correspond to structural asymmetries that may require further guarding:

- Controlled operations without stabilisation (Ω without Ψ) can become **structurally unstable unless further guarded** if they escalate system-wide imbalance.
- High-complexity asymmetries may require an attractor or stabiliser commitment (A or Ψ).

$\Omega(S)$ without $(A \vee \Psi)$ is structurally unstable unless further guarded.

Integration Constraints (Σ-boundaries)

- A Σ boundary commits previous transformations.
- Any reframing (Φ) after Σ must treat Σ as an invariant boundary.

$\Sigma(E) \rightarrow$ no reinterpretation of E except under Ψ.

These constraints reflect praxeological principles but are expressed here purely structurally, independent of interpretive content.

5.4 Structural Constraints Are Not Physical Guarantees

The structural constraints introduced above should not be mistaken for hardware guarantees, safety guarantees, correctness proofs, or validated compiler rules. They express how Δ–Ψ compositions are organised and bounded **within this framework**, not how every concrete quantum system will behave under physical execution.

Accordingly:

- a structurally licensed macro chain may still be physically inadequate, noisy, or poorly compiled,
- a structurally unstable chain may still be mathematically interesting or experimentally useful in a bounded context,
- and a governance or stabilisation annotation does not by itself secure safe deployment, fault tolerance, or implementation adequacy.

This distinction is especially important because the macro language is intended to inform later YAML policy layers. Structural expressiveness must therefore not be confused with physical sufficiency.

5.5 Summary

The praxeological composition language provides a structural abstraction of quantum circuits in terms of Δ–Ψ operators. Macro-operators capture typical patterns of quantum computation—initialisation, oracle marking, amplitude amplification, iteration, measurement and stabilisation—while the grammar and constraints formalise permissible compositions at a structural level. This establishes PMS-QC as a workable object-language for describing quantum algorithm structure, preparing the ground for operator-level reconstructions of canonical quantum algorithms, without claiming exhaustive or physically sufficient capture of QC composition.

6. Grover's Algorithm as a Praxeological Structure

Grover's search algorithm is a canonical example of a quantum procedure whose structure is dominated by repeated asymmetry, attractor dynamics and stabilised iteration. In this section, we reconstruct Grover's algorithm using the praxeological composition language introduced above, showing how its components can be rendered as Δ–Ψ operator sequences.

The reconstruction presented here distinguishes between the **standard Grover iteration**, which is fully expressible within the generic PMS-QC base layer, and **optional stabilised or fixed-point variants**, which rely on additional Ψ-based constraints introduced in the PMS-QC_EXT research layer.

This chapter should be read as a demonstration of **structural legibility**, not as a claim to the exclusive or deepest possible truth of Grover's algorithm, and not as a rival description displacing standard QC theory.

6.1 Classical Structure of Grover's Algorithm

Given an oracle O_F marking a unique solution $x^* \in \{0,1\}^n$, Grover's algorithm consists of:

1. **Initialisation:**
   Uniform superposition over all 2^n computational basis states.
2. **Oracle application:**
   Application of a phase oracle O_F that flips the phase of x^*.
3. **Diffusion operator:**
   Inversion about the mean, amplifying amplitude at the solution state.
4. **Iterations:**
   Repeating the oracle–diffusion block $k \approx \lfloor(\pi/4)\sqrt{(2^n)}\rfloor$ times.
5. **Measurement:**
   Reading out the amplified solution.

This structure can be rendered especially clearly within the Δ–Ψ grammar.

6.2 Praxeological Reconstruction Using Macro-Operators

Step 1 — Frame Initialisation

```
QFRAME(problem)
```

Induces a □-frame capturing the search space and oracle domain:

$\Box(S_0)$.

Step 2 — Preparation of the Δ-Field

```
QPREP(superposition)
```

Introduces Δ-distributed alternatives across all basis states:

$A(\nabla(\Delta(\square(S_0))))$.

The resulting state is a uniform Δ-field: all alternatives are distinguishable but not yet asymmetrically marked.

Step 3 — Iterative Structure

Grover's core loop is expressed praxeologically as:

```
QITERATE(k, { QORACLE_CALL(F); QDIFFUSION; })
```

Corresponding Δ–Ψ decomposition:

$\Theta^k(\Sigma(A(\Omega(\Delta(\square(S))))))$.

This operator chain captures the **standard Grover iteration** as a generic PMS-QC structure. No self-binding (Ψ) constraint is required at this level; iteration depth (k) is treated as an external parameter rather than as a structurally enforced invariant.

Breaking this down:

- **Oracle call** applies Ω to Δ-structures inside the active □-frame, marking the distinguished element x*.
- **Diffusion** applies A to create an attractor toward x*, followed by Σ to integrate the attractor update.
- **Iteration** Θ repeats the composed structure exactly k times.

The sequence

$\Omega \rightarrow A \rightarrow \Sigma$

provides a compact Δ–Ψ rendering of “mark → amplify → commit”.

Step 4 — Measurement

```
QMEASURE(basis=computational)
```

Implements:

$\Lambda(\Delta(S))$.

Measurement distinguishes realised from unrealised alternatives (Δ), while Λ encodes the non-occurring branches.

6.3 Structural Interpretation

Grover's algorithm yields several insights when expressed in PMS terms.

(A) Distributed Δ-field and Ω-marking

The algorithm begins with a maximally distributed Δ-field (uniform superposition). Ω then introduces a structural asymmetry by distinguishing the target state. This mirrors a generic praxeological pattern:

- Δ: multiple viable alternatives
- Ω: asymmetry imposed by an informational constraint (oracle knowledge)

(B) Attractor Dynamics (A) and Integration (Σ)

Amplitude amplification becomes:

A → Σ.

A introduces directional convergence; Σ commits the attractor effect. Each Grover iteration strengthens the attractor and integrates it into the structure.

(C) Temporal Regularity (Θ)

The repeated block corresponds to Θ, the operator of temporal iteration. The iteration is fixed-length, not adaptive, which aligns with Θ's role as a structurally regular, pre-specified sequence.

(D) IA-Type Patterns (Structural Analogy Only)

Grover's oracle structure exhibits a structurally relevant asymmetry:

- The oracle “knows” the target; the superposition “does not”.
- The computation creates an alignment that can be compared, at a purely formal level, to the pattern $IA_{A \gg E}$ (asymmetric awareness-to-action alignment).

This comparison should be read strictly as a **structural analogy**. It does not anthropomorphise the algorithm, does not attribute genuine awareness or agency to quantum states, and does not license an unmarked transfer of anthropological PMS derivations into the QC domain. The narrower point is simply that Ω-marking creates a formally asymmetric relation between distributed alternatives and selective action.

(E) Stabilisation via Ψ (Optional and EXT-Bound)

Grover's algorithm in its **standard form** does not include an explicit Ψ-operator. Any Ψ-based stabilisation—such as overshoot prevention, fixed-point convergence, or iteration guards—constitutes a **paper-specific refinement** and is therefore assigned to the optional PMS-QC_EXT layer.

- A Ψ layer could enforce a stop condition when amplitude amplification enters a stability basin.
- Fault-tolerance or coherent iteration guards can be rendered as Ψ-type invariants ensuring that Θ-repetition remains within bounded error conditions.

Formally, a stabilised version would include:

$\Psi(\Theta^k(E))$,

interpreting Ψ as enforcing bounds on iteration depth, residual phase error or circuit coherence.

Nothing in the generic PMS-QC base representation requires such a Ψ layer. Standard Grover remains a base-layer construction; fixed-point or overshoot-regulated versions remain EXT-only.

6.4 Summary

Grover's algorithm maps clearly onto the PMS operator grammar:

- □ establishes the computational frame
- **Δ / Ω** mark structural alternatives
- **A / Σ** implement amplitude-based attractor dynamics
- **Θ** provides iterative structure
- **Λ** encodes realised vs. unrealised outcomes
- **Ψ** provides an optional structural locus for stabilisation under paper-specific refinement

This reconstruction demonstrates that quantum algorithm structure can be expressed clearly and compactly using the Δ–Ψ operator family, setting the stage for analysing a second, structurally distinct quantum algorithm.

All Ψ-based stabilisation mechanisms discussed in this section are optional refinements and must not be interpreted as requirements of the generic PMS-QC representation of Grover's algorithm.

7. Second Example: Quantum Phase Estimation as a Praxeological Structure

Before introducing algorithm-specific structures, we clarify the role of **policies and constraints** in the PMS-based analysis. All policies discussed in this section operate at the level of **structural governance and stabilisation**. They constrain admissible compositions of Δ–Ψ operator chains but do **not** guarantee algorithmic correctness, numerical precision, physical stability, or performance characteristics of a concrete quantum implementation.

To demonstrate that the PMS operator grammar extends beyond search-based amplitude amplification, we analyse *Quantum Phase Estimation* (QPE), a canonical algorithm for extracting eigenphases of a unitary operator. QPE differs structurally from Grover's algorithm in that it relies on controlled unitaries, multi-level asymmetry structures, fine-grained temporal iteration and a non-iterative attractor mechanism via the inverse Quantum Fourier Transform (QFT). These features introduce rich Ω, Θ, A configurations and possible Ψ placements.

From the outset, one boundary must remain explicit: the power-of-two ladder U^{2^j} discussed below is treated in this paper as a **paper-specific macro refinement**, not as a generic PMS-QC base rule.

7.1 Standard Structure of Quantum Phase Estimation

Given:

- a unitary U with eigenstate $|\psi\rangle$,
- such that $U|\psi\rangle = e^{2\pi i\varphi}|\psi\rangle$,

QPE estimates the phase φ by the following steps:

1. **Initialisation:** Prepare a control register of t qubits in the uniform superposition and a target register in eigenstate $|\psi\rangle$.
2. **Controlled unitary sequence:** Apply U^{2^j} to the target, controlled by the j-th control qubit.
3. **Inverse QFT:** Apply QFT^{-1} to the control register.
4. **Measurement:** Measure the control register in the computational basis to obtain an estimate of φ.

This structure is more nuanced than the Grover iteration and is therefore well-suited for an additional demonstration of structural portability within this paper.

From a layering perspective, we distinguish between **generic structural constraints** that belong to the PMS-QC base layer and **paper-specific refinements** introduced in PMS-QC_EXT. Generic policies capture generally reusable frame, asymmetry and integration constraints, whereas PMS-QC_EXT policies address algorithm-specific sensitivity, precision and stabilisation issues particular to QPE.

7.2 Praxeological Reconstruction Using Macro-Operators

Step 1 — Frame for the Eigenproblem

```
QFRAME(eigenproblem)
```

$\Box(S_0)$.

This defines the structural frame representing the composite Hilbert space of control and target registers, scoped to the eigenproblem of U.

Step 2 — Preparation of the Δ-Field

```
QPREP(superposition)
```

Applied to the control register, this produces a distributed Δ-field over computational basis states:

$A(\nabla(\Delta(\Box(S_0))))$.

The eigenstate in the target register acts as an embedded subframe, not requiring Δ-preparation.

Step 3 — Controlled Unitary Ladder (Ω/Θ)

The key body of QPE is a sequence of controlled operations $C\text{-}U^{2^j}$. Structurally:

```
QITERATE(t, QORACLE_CALL(U^{2^j}))
```

but now the "oracle" is a controlled unitary, not a selective marker. In Δ–Ψ terms:

$\Theta^t(\Omega(\Box(S)))$.

The exponential controlled-unitary ladder U^{2^j} is treated here as a **paper-specific macro refinement**. While its Δ–Ψ structure $\Theta^t(\Omega \circ \Box)$ provides a compact structural description, the ladder itself is **not** elevated to a generic PMS-QC rule and is later formalised explicitly in the optional PMS-QC_EXT layer.

Here:

- **Ω** represents the controlled asymmetry: the target evolves conditionally on the control register.
- **Θ** sequences these operations in increasing power-of-two steps, establishing a structured temporal ladder.

This block is a paradigmatic instance of high-order Ω–Θ coupling.

Step 4 — Inverse QFT as Φ/A with Σ-Commit

The inverse Quantum Fourier Transform (QFT^{-1}) transforms the control register from a phase-encoded Δ-distribution into a basis-aligned attractor around the binary expansion of φ. Structurally, this operation is **not diffusion**, but a **reframing followed by Fourier-domain alignment**, with an explicit integration step.

This is captured by the dedicated macro:

```
QFT_ALIGN(inverse) Σ
```

A concise Δ–Ψ expression is:

Σ(A(Φ(S))).

Interpretation:

- **Φ**: reframes the register from the Fourier (phase) domain back into the computational basis.
- **A**: aligns amplitudes in the reframed domain, concentrating probability mass around the correct phase estimate.
- **Σ**: commits the aligned distribution as a stable structure for subsequent measurement.

This ordering fixes the semantics unambiguously: Φ acts first, then A, and Σ commits the result. It also keeps **QDIFFUSION** reserved for Grover-style iterative amplitude amplification and avoids conflating diffusion with Fourier-based reframing.

Step 5 — Measurement

```
QMEASURE(basis=computational)
```

Expands to:

Λ(Δ(S)).

This collapses the phase-encoded distribution to the nearest binary approximation of φ.

7.3 Structural Interpretation

All Ψ placements discussed here should be understood as **possible structural loci of stabilisation**, not as requirements of Quantum Phase Estimation. These constraints belong either to the generic PMS-QC governance layer (e.g. frame coherence) or to paper-specific PMS-QC_EXT refinements addressing precision and noise sensitivity in QPE.

(A) Multi-Level Asymmetry (Ω)

Controlled U^{2^j} operations instantiate *hierarchically scaled asymmetries*:

- The control qubit "acts on" the target with increasing influence (powers of two).
- Ω is therefore *graded*, not binary.

This contrasts with Grover's fixed Ω-marking.

(B) Fine-Grained Temporal Structure (Θ)

The Θ-sequencing in QPE is more intricate:

- It encodes exponential scaling of unitary powers.
- The ladder-like structure is a canonical instance of *structured temporal asymmetry*.

(C) Reframing (Φ) as Interpretive Transition

Φ plays a central role:

- The inverse QFT reinterprets the structural landscape from phase space to bit space.
- Φ thereby captures the conceptual reframing required to extract a classical approximation.

Grover's algorithm contains no such reframing.

(D) Attractor Formation Without Iteration (A)

Unlike Grover's iterative attractor dynamics, QPE employs a *single* attractor transformation (QFT^{-1}):

$A \to \Sigma$.

This highlights A's generality:

- Sometimes A is iterative (Grover).
- Sometimes A is non-iterative but high-dimensional (Fourier alignment).

(E) Stabilisation and Ψ-Placement

QPE is more sensitive to precision and noise than Grover. A PMS-level analysis suggests multiple **possible structural loci** for Ψ:

- Ψ enforcing coherence constraints for the controlled-unitary ladder.
- Ψ bounding phase-estimation error after Σ.
- Ψ stabilising the frame during Φ (since incorrect reframing is catastrophic).

QPE therefore provides a clear site for describing stabilisation options within this grammar, without implying that any single placement is uniquely natural, optimal, or required.

7.4 Ladder Refinement Without Base Inflation

The controlled ladder U^{2^j} is one of the clearest places where paper-specific refinement must be kept distinct from the generic PMS-QC base layer.

Its structural rendering,

$\Theta^t(\Omega \circ \square)$,

is perfectly compatible with the base grammar. But the ladder as a named, reusable macro-pattern remains **paper-specific** because it encodes a more specialized algorithmic

organisation than the generic base macros are meant to capture.

For that reason:

- `QORACLE_CALL` remains the generic base macro for asymmetry-bearing oracle or controlled-unitary action,
- the explicit `U^(2^j)` ladder belongs only to the optional EXT layer,
- and the existence of this refinement does not alter the semantics of the base macro language.

This separation matters methodologically. It prevents useful paper-derived structure from being silently promoted into the base specification, and it keeps the PMS-QC base layer reusable for broader QC description without inflating it with algorithm-specific templates.

7.5 Summary

Quantum Phase Estimation demonstrates that the Δ–Ψ grammar extends beyond search-based algorithms:

- □ encapsulates the eigenproblem frame
- **Δ / Α** encode superposition and phase-structured attractors
- **Ω / Θ** produce scaled, ordered control asymmetries
- **Φ** enables interpretive reframing
- **Σ** commits the Fourier-aligned structure
- **Λ** captures unrealised alternatives via measurement
- **Ψ** provides possible structural loci for stabilisation under noise and precision limits

QPE thus serves as an additional demonstration of portability within this paper, showing that the PMS grammar can describe qualitatively different quantum algorithmic architectures without thereby establishing universal closure or unique adequacy.

All policies and constraints introduced in this section are structural in nature. They regulate admissible operator compositions and stability conditions but make no claims about physical correctness, convergence guarantees, or computational efficiency of concrete QPE implementations.

8. PMS as Control-Layer for Agentic Quantum Computing Systems

As quantum computing systems increasingly integrate machine-learning models and autonomous agents—especially large language model (LLM)-based controllers—the need for structured governance and compositional safety grows. Such systems often involve an agent deciding *which* quantum routines to run, *under which conditions*, and *with which parameters*, based on high-level objectives or adaptive reasoning. This creates a multi-layered architecture in which quantum circuits, classical computation and agentic decision processes may interact.

Praxeological Meta-Structure Theory (PMS) can be used, under bounded conditions, as a structural grammar for describing and constraining such hybrid systems. By expressing both quantum workflows and agentic decisions as Δ–Ψ operator chains, PMS may provide a basis for cross-domain auditability, structural constraint enforcement and policy-guided action selection. This should not be read as implying that every PMS-QC description naturally culminates in governance, nor that a control-layer interpretation is the necessary completion of the framework.

A strict separation between the PMS base grammar, the generic PMS-QC structural layer, and optional paper-specific extensions is essential for maintaining semantic stability and reusability. Layering prevents meaning drift by ensuring that the Δ–Ψ operators remain invariant, while domain-specific interpretations and constraints are isolated at higher levels. This allows the PMS-QC base to function as a reusable, algorithm-agnostic structural layer, while research-driven refinements can be introduced, evaluated and discarded without affecting the underlying operator semantics.

8.1 Governance Conditionality / Non-Necessity of Control-Layer Handling

The governance-oriented reading developed in this chapter is a **conditional architectural option**, not the necessary endpoint of PMS-QC. One may use PMS-QC purely for structural description of quantum algorithms, for macro-level comparison of circuit organisation, or for paper-specific refinement work, without moving into policy-layer or control-layer applications at all.

Accordingly, the present chapter should be read as exploring one important extension path: how PMS-QC **may** be used to articulate auditability, constraint management and bounded coordination in hybrid AI–QC settings. It does not claim that governance is the natural telos of the theory, nor that formal policy expressiveness by itself establishes a deployable or normatively sufficient control system.

Application Boundary Note

Nothing in this chapter should be read as licensing clinical, personal, or moral interpretations of QC actors or hybrid systems. Likewise, the language of agency, governance, or constraint should not be taken to anthropomorphise quantum processes or to imply silent promises of safety, capability, or deployment readiness.

8.2 PMS as an Audit Layer

In hybrid systems, agentic control flows produce complex compositions of quantum and classical operations. PMS can be used as an operator-level audit mechanism through:

- **Structural replay:** Quantum workflows can be lifted into Δ–Ψ macro-operator sequences (e.g., `QFRAME`, `QPREP`, `QORACLE_CALL`).
- **Ex post inspection:** PMS examines whether the sequence obeys composition constraints (e.g., frame continuity, permissible asymmetries, valid Σ-boundaries).
- **Structural diagnostics:** Potential violations—e.g., unframed Ω, reframing after Σ without Ψ, or unstable A→Σ sequences—can be detected by analysing the Δ–Ψ structure rather than low-level gates.

This may provide a disciplined governance aid without modifying the underlying quantum hardware or low-level software stack. It does not, however, amount by itself to a validated safety system, comprehensive audit guarantee, or deployment-ready governance framework.

8.3 PMS as Execution Policy Layer

PMS can also be used to *describe and constrain* agentic behaviour by defining admissible Δ–Ψ sequences. The resulting policy vocabulary should be understood as **structural governance heuristics and constraints**, rather than as a self-sufficient normative safety architecture.

Policy Classes

Examples of structural policies include:

- **Frame policies:** `no_unframed_omega` — disallow asymmetry operators (Ω) unless a proper □-frame is active.
  `measurement_requires_meas_frame` — require a measurement-stable frame before invoking `QMEASURE`.
- **Integration policies:** `no_reframing_past_sigma` — prohibit Φ-based reframing after a Σ commit boundary unless explicitly stabilised.
  `unstable_asymmetry_guard` — constrain Ω-heavy compositions unless coupled with attractor (A) or stabilisation (Ψ) mechanisms.
- **Self-binding policies:** `max_iteration_depth` — enforce an upper bound on Θ-expansion (iteration depth).
  `stabiliser_frequency` — require periodic Ψ application in long-running or high-noise workflows.
- **Reframing policies:** Explicit Φ transitions must be declared when switching between computational, physical, or governance frames, and are subject to the above integration and stabilisation constraints.

These policies may be instantiated as YAML-level rule identifiers in the PMS-QC specification and may serve as operational guardrails for auditing, compilation and agentic control. They extend natural QC constraints with praxeological structure, enabling fine-grained and machine-checkable control over algorithmic composition and agent actions. But policy expressiveness here should not be confused with deployment adequacy, verified safety, or sufficient runtime control in real systems.

8.4 Multi-Frame Agents and Domain Coordination

Hybrid systems operate across multiple conceptual domains:

- a **physical frame** (quantum hardware, noise model),
- a **computational frame** (circuits, logical qubits),
- a **business or task frame** (problem specification, objective function),
- a **governance frame** (constraints, risk models, policies).

PMS provides operators for coordinating these frames:

- **Φ (Reframing):** Transitions between frames, such as switching from solution-finding to verification or from physical considerations to logical ones.
- **X (Distancing):** Domain isolation, e.g., separating exploratory agent behaviour from production-critical QC runtimes.
- **Σ / Ψ (Integration & Self-binding):** Consolidating validated results and stabilising long-term policies that must persist across frames.

This may provide a more structured and transparent way of representing multi-frame operation without conflating physical, computational and governance constraints. Even so, formal structural description should not be confused with a secure agent architecture: a well-described multi-frame system is not, by that fact alone, a safe or validated one.

8.5 Prompt and Inference Control

If the controlling agent is an LLM or LLM-based composite, PMS may serve as a **possible structuring interface for bounded experimental architectures** rather than as a fully established prompt- or inference-governance solution.

- **Δ / □** provide a structural description of the *input space* the agent may act upon. Frames can define allowable prompt contexts or restrict the semantic domain. Δ describes distinctions among admissible actions.
- **Σ / Ψ** provide a way of expressing persistent constraints on the agent's inference behaviour:
  - Σ commits validated control strategies or workflow templates.
  - Ψ enforces long-term invariants (e.g. “never run unverified QPE”, “do not exceed a coherence budget”, “respect inter-frame isolation”).

This allows one to interpret prompting and inference as praxeological action within explicitly governed Δ–Ψ structures. But it does not show that PMS already yields a robust, sufficient, or deployment-ready governance architecture for LLM-based QC orchestration.

8.6 PMS as a Meta-Compiler Layer

PMS can be placed above existing QC frameworks (Qiskit, Cirq, Braket) as a meta-compiler layer:

1. A high-level specification (from an agent or human) is expressed in Δ–Ψ macro-operators.
2. PMS checks the structure against policy constraints.
3. Valid PMS expressions are compiled into backend-compatible circuits.
4. Execution traces are lifted back to Δ–Ψ sequences for auditing and further reasoning.

In effect:

- The QC framework handles **physical correctness**.
- PMS handles **structural correctness**.
- The agent handles **goal-directed reasoning**.

This division of labour may be useful as an architectural separation proposal for future quantum–agent systems. It is not, however, a validated deployment template. A meta-compiler layer described in PMS terms does not by itself establish safe execution, reliable orchestration, or production-worthy control.

8.7 Summary

Within bounded conditions, PMS can be used as a candidate structural control layer for hybrid QC/AI systems:

- **As audit layer:** analysing Δ–Ψ compositions of quantum workflows.
- **As execution policy:** constraining permissible operator sequences and asymmetries through structural governance heuristics.
- **As multi-frame coordinator:** structuring transitions between physical, logical, task and governance domains.
- **As prompt/inference interface:** expressing agent constraints using Σ/Ψ invariants in bounded experimental settings.
- **As meta-compiler layer:** validating and translating high-level praxeological programs into backend QC implementations.

This chapter therefore offers a candidate architectural path under bounded conditions, not a final or uniquely privileged governance architecture for emerging agentic quantum systems.

9. Discussion

The application of Praxeological Meta-Structure Theory (PMS) to quantum computing raises several conceptual and technical questions concerning expressive power, portability and scope. This section discusses how PMS relates to existing formalisms, what its substrate-independent character makes possible within the present framework, and where its limits lie.

9.1 Expressive Power of the Δ–Ψ Operator Grammar

The PMS operator system offers a compact and interpretable grammar for describing structured action across domains. Its expressive power derives from several properties:

- **Minimality:** Δ–Ψ consists of only eleven irreducible operators, yet captures differentiation, framing, asymmetry, iteration, attractor dynamics, integration and self-binding.
- **Compositionality:** The operator calculus supports hierarchical composition, enabling multi-level structures such as nested frames, iterated asymmetries and stabilised workflows.
- **Cross-domain abstraction:** Because the operators act on arbitrary structural states S, the calculus can be used across praxeological, computational and physical settings as a cross-domain portability claim within the PMS framework.

Although PMS can express many of the structural features of quantum circuits, it is not in competition with quantum mechanical formalisms such as the circuit model, Hamiltonian simulation, ZX-calculus or categorical quantum mechanics. Instead, it provides an **external structural language** for describing the organisation of operations, not their underlying physics.

9.2 Portability of PMS Across Substrates

One of PMS's key motivations is to provide an operator grammar that can be applied across otherwise unrelated domains. The Δ–Ψ operator family is deliberately substrate-independent:

- In **praxeological contexts**, Δ marks distinctions, Ω expresses asymmetry, Σ commits and Ψ binds.
- In **software architectures**, Δ captures branching structures, Θ loops, Φ context shifts and Ψ invariants.
- In **agentic systems**, frames (□) isolate domains, Ω expresses role-based decision asymmetries and Σ/Ψ encode policy commitments.
- In **quantum computing**, Δ maps to distributed superposition, Ω to controlled operations, A to amplitude alignment, Σ to diffusion or QFT integration, and Ψ to error-correction constraints.

The same operator vocabulary can therefore describe heterogeneous processes using a single structural syntax. This does not imply equal explanatory depth in all domains, nor does it imply exhaustive capture. Its portability is not based on reducing everything to the same mechanics but on identifying **isomorphic structural roles** across systems of action.

9.3 Substrate Independence as a Structural Principle

PMS emphasises that many systems—physical, computational, organisational, agentic—exhibit structurally comparable processes:

- **Framing (□):** Hilbert spaces, program contexts, organisational boundaries, task domains.

- **Asymmetry (Ω):** Controlled gates, access privileges, informational asymmetries, role differentiation.
- **Iteration (Θ):** Loops, repeated updates, periodic processes in physics and computation.
- **Self-binding (Ψ):** Stabiliser codes, invariants in software, commitments in agent policies, norms in organisational practice.

This supports the idea that PMS captures a structural form of substrate independence: a single operator system that can describe compositional logic across domains despite differing local semantics. Structural comparability here should not be confused with ontological identity: that two domains can be rendered through the same operator vocabulary does not mean that they are the same kind of thing.

9.4 Computational Implications

The Δ–Ψ framework suggests several avenues for computational analysis, but these should be distinguished carefully from claims of actual validation or predictive proof:

- **Structural complexity heuristics:** PMS expressions could be used to classify quantum algorithms by structural depth (number of Θ iterations), asymmetry density (Ω frequency), or stabilisation strength (Σ/Ψ placement).
- **Comparative operator metrics:** Algorithms may be compared by how they deploy A-type dynamics, whether iterative (Grover) or single-shot (QPE).
- **Constraint-localisation:** Ψ may provide a principled location for expressing stop conditions, error bounds or coherence constraints at a structural level.
- **IA-risk heuristics:** While not treating quantum algorithms anthropologically, structural analogues of asymmetry–action imbalances (IA-patterns) may indicate loci of instability or sensitivity in compositional design.

These implications point to PMS as a potential tool for high-level reasoning about algorithmic structure, independent of low-level gate semantics. They should be read as analytical and heuristic possibilities; actual validation still depends on domain-specific mathematical, empirical and implementation-level work.

9.5 Limitations

Despite its expressive power, PMS has clear boundaries:

1. **Not a replacement for quantum formalism:** PMS does not model unitaries, amplitudes or decoherence; it only describes their structural organisation.
2. **Abstraction risk:** There is a risk of over-interpreting anthropological terminology when applied to technical systems. The Δ–Ψ operators must be treated *strictly as formal structures*, not as metaphors.
3. **No predictive guarantees:** PMS does not yield performance predictions or correctness proofs; it complements but does not substitute formal verification or physical modelling.
4. **Granularity mismatch:** PMS operates at a higher level of abstraction than gate-level quantum models; some fine-grained quantum effects have no direct Δ–Ψ analogue.
5. **Validation scope:** PMS describes structure, not mechanism; empirical performance must still be evaluated within domain-specific frameworks.
6. **Extension-layer limitations:** Optional extensions such as *PMS-QC_EXT* are intentionally

non-universal and non-binding. They do not modify the Δ–Ψ operator grammar, nor are they required for applying PMS or PMS-QC. Instead, such extensions capture paper-specific refinements, governance heuristics or macro-level constraints that may be useful in particular research contexts but are neither exhaustive nor generally valid beyond their stated scope.

7. **Non-capture remains possible:** Even where structural rendering succeeds, some relevant distinctions may remain only partially visible or may be foregrounded more effectively by rival or complementary formalisms.

Overall, PMS is a *meta-language* rather than a computational model: it provides a strong conceptual framework for structural composition, but it does not alter or augment the physics of quantum computation.

9.6 Scope Limit of Structural Legibility in QC

Structural legibility is a real strength of PMS-QC, but it has limits.

First, **not everything in quantum computing becomes more visible under PMS**. Certain low-level, quantitative, or hardware-sensitive features are better captured by domain-specific formalisms closer to the physics or mathematics of implementation.

Second, **rival or complementary formalisms may foreground different distinctions more effectively**. ZX-calculus, categorical methods, Hamiltonian approaches, or verification-oriented frameworks may remain better suited where the central task is algebraic simplification, physical modelling, optimisation, or formal proof rather than cross-domain operator rendering.

For that reason, the value of PMS-QC lies not in replacing alternative frameworks, but in contributing a disciplined structural viewpoint whose strength is portability, bounded abstraction and operator-level comparability.

10. Conclusion

This paper introduced Praxeological Meta-Structure Theory (PMS) as a substrate-independent operator system and demonstrated, within the scope of this paper, its applicability to the compositional analysis of quantum algorithms. PMS provides, within the present framework, a minimal and irreducible set of operators (Δ–Ψ) capable of expressing differentiation, framing, asymmetry, iteration, attractor dynamics, integration and self-binding. As such, it offers a portable cross-domain grammar for describing structured action across heterogeneous systems.

Quantum computing constitutes an especially suitable testbed for examining this operator system. Quantum circuits are highly structured, strongly compositional and sensitive to asymmetry, temporal iteration and stability—properties that align closely with the Δ–Ψ calculus. By reconstructing Grover's search algorithm and Quantum Phase Estimation within this framework, we showed that PMS can express both iterative attractor dynamics and non-iterative phase-alignment structures, illustrating its portability across qualitatively different quantum architectures.

The core result is that, within the present framework, PMS provides a portable structural language capable of relating praxeological theory, AI- and agent-based architectures, and quantum algorithms within a single operator grammar. The Δ–Ψ system does not replace quantum formalisms; instead, it complements them by offering a meta-theoretical layer for reasoning about composition, constraint and governance. This same grammatical structure can be applied to agentic decision processes, multi-frame environments and cross-domain interactions, while leaving open questions of comparative adequacy, boundedness and non-capture.

10.1 Strength Without Finality

The argument of this paper is intentionally strong, but it is not a closure claim. Nothing here establishes completeness, unique adequacy, or immunity to rival formalisation. The fact that PMS can render important QC structures in a disciplined and portable way does not show that every relevant distinction in quantum computing is best captured by this grammar, nor that rival frameworks could not outperform it for other explanatory tasks.

The strongest conclusion available here is therefore methodological: PMS-QC provides a coherent and reusable structural proposal for cross-domain operator analysis. Its strength lies in the combination of portability, compositional clarity and bounded formal discipline, not in a claim to final interpretive authority.

Several directions for future work remain open. All paper-specific refinements discussed here are deliberately isolated in an optional extension layer (PMS-QC_EXT), ensuring that the core PMS operator grammar and the generic PMS-QC layer remain stable, reusable and independent of individual research contexts. Possible next steps include **compiler-level integration**, through which PMS-annotated quantum programs might be validated and translated into backend circuits; **agentic governance layers**, through which PMS operators may be used to define admissible action policies, stabilisation rules and frame transitions in

multi-agent QC environments; and possible forms of **standardisation**, through which PMS-QC could contribute to AI–QC structural reasoning without displacing existing quantum models.

Taken together, these results position PMS not as a final universal operator calculus, but as a strong and portable structural proposal for articulating the organisation of quantum computation, agent systems and praxeological processes within a single coherent framework.

Appendix A — Paper / Base / EXT Boundary Map

This appendix documents the boundary between the **generic PMS-QC base overlay**, the **optional PMS-QC_EXT refinement layer**, and **paper-level explanatory prose**. Its purpose is methodological: to prevent silent drift between the main paper, `pms-qc.yaml`, and `pms-qc-ext.yaml`, and to ensure that paper-specific refinements are not retroactively misread as base semantics.

Nothing in this appendix modifies PMS, PMS-QC, or PMS-QC_EXT. It records the intended interpretive and specification boundary used in this paper.

A.1 Boundary Principle

The working rule is:

- **Base (PMS-QC):** what remains generically usable as a QC structural layer even without the present paper's more specialised reconstructions.
- **EXT (PMS-QC_EXT):** what depends directly on paper-specific algorithm reconstructions, stabilisation refinements, or stronger governance heuristics.
- **Paper-only:** explanatory, comparative, or methodological prose that clarifies how the operator grammar is being used, but does not itself constitute a reusable macro, policy, or schema element.

This separation should be read together with the PMS 1.3 repository discipline: overlays may refine and organise, but must not silently redefine the PMS operator set, dependency spine, or base-layer semantics.

A.2 Boundary Table

Item	Status	Layer	Reason
Δ–Ψ operator meanings	Canonical reference only	PMS base	Defined in `PMS.yaml`, not in this paper or QC overlays
PMS dependency spine	Canonical reference only	PMS base	Must remain subordinate to PMS 1.3
QC operator mappings (Δ–Ψ → QC roles)	Generic structural rendering	PMS-QC base	Algorithm-agnostic overlay mapping
`QFRAME`	Generic macro	PMS-QC base	Reusable structural framing macro
`QPREP`	Generic macro	PMS-QC base	Reusable preparation macro
`QORACLE_CALL`	Generic macro	PMS-QC base	Captures oracle / controlled-unitary asymmetry without ladder-specific refinement
`QDIFFUSION`	Generic macro	PMS-QC base	Captures attractor + integration structure without fixed-point guarantees
`QFT_ALIGN`	Generic macro	PMS-QC base	Captures reframing + alignment without paper-specific QPE refinement
`QITERATE`	Generic macro	PMS-QC base	Generic Θ-structured repetition
`QMEASURE`	Generic macro	PMS-QC base	Generic structural resolution of outcomes
`QSTABILIZE`	Generic macro	PMS-QC base	Generic Ψ-location for invariants or stabilisation

Item	Status	Layer	Reason
`no_unframed_omega`	Generic policy	PMS-QC base	Reusable frame discipline
`measurement_requires_meas_frame`	Generic policy	PMS-QC base	Reusable measurement-frame discipline
`unstable_asymmetry_guard`	Generic policy	PMS-QC base	Generic structural stability heuristic
`no_reframing_past_sigma`	Generic policy	PMS-QC base	Generic Σ-boundary discipline
`max_iteration_depth`	Generic policy	PMS-QC base	Generic iteration-bounding policy
`stabiliser_frequency`	Generic policy	PMS-QC base	Generic stabilisation-density policy
Standard Grover reconstruction	Generic structural rendering	PMS-QC base	Fully expressible without EXT refinement
Standard Grover without Ψ	Explicit base reading	PMS-QC base	Paper insists standard Grover does not require Ψ
Fixed-point Grover	Paper-specific refinement	PMS-QC_EXT	Requires additional Ψ-bound stabilisation logic
Overshoot-regulated Grover	Paper-specific refinement	PMS-QC_EXT	Not part of standard Grover base rendering
`GROVER_CENTERING`	Refinement macro	PMS-QC_EXT	Specialised diffusion-centering refinement
`grover_no_overshoot`	Refinement policy	PMS-QC_EXT	Algorithm-specific guard, not generic base policy
Minimal QPE rendering	Generic structural rendering	PMS-QC base	Base skeleton is allowed without promoting ladder to base
`QORACLE_CALL(U^(2^j))` in base example	Placeholder only	PMS-QC base example	Allowed as minimal rendering, but not as reusable ladder macro
Explicit `U^(2^j)` ladder as reusable pattern	Paper-specific refinement	PMS-QC_EXT	Deliberately reserved for EXT
`QPE_ORACLE_LADDER`	Refinement macro	PMS-QC_EXT	Specialized QPE ladder construct
`qpe_depth_guard`	Refinement policy	PMS-QC_EXT	QPE-specific ladder-depth guard
QPE Ψ placements	Possible structural loci only	Paper prose / PMS-QC_EXT	Discussed as optional sites, not base requirements
`Φ → A → Σ` ordering for QPE alignment	Editorial semantic ordering	PMS-QC base plus paper prose	Needed for base consistency, but described in paper prose and reflected in YAML examples
`ATTRACTOR_FIXED_POINT`	Refinement macro	PMS-QC_EXT	Paper-specific fixed-point schedule template
`MEASUREMENT_PROJECTION_MODEL`	Refinement macro	PMS-QC_EXT	Specialized branching-model refinement
Governance conditionality	Methodological constraint	Paper prose and YAML guard language	Not a macro or policy by itself; governs interpretation
Application Boundary Note	Methodological guard	Paper prose / YAML prose	Boundary statement, not executable schema logic
IA-analogy in Grover	Structural analogy only	Paper prose	Explicitly not a schema primitive or policy
Meta-compiler humility clause	Methodological guard	Paper prose / YAML prose	Clarifies non-deployment status

Item	Status	Layer	Reason
Cross-domain portability claim	Methodological framing	Paper prose / YAML prose	Not a reusable operator or macro
Non-identity clause	Methodological framing	Paper prose / YAML prose	Guards against ontological overread
Non-guarantee clause	Methodological framing, attached to macros/policies	PMS-QC base and PMS-QC_EXT	Required throughout, but not a new operator

A.3 Boundary Implications

Three consequences follow from this map.

First, **the existence of a paper-specific refinement does not retroactively enrich the base layer**. The fact that QPE can be described more precisely with a ladder refinement does not mean that the ladder belongs to the generic semantics of `QORACLE_CALL`.

Second, **base examples remain intentionally skeletal**. Their role is to show that a generic PMS-QC description is possible without silently importing paper-specific constructs into the base overlay.

Third, **methodological guards are not disposable prose decorations**. Clauses such as non-identity, non-guarantee, governance conditionality, and application boundary notes are part of the disciplined reading of the schema family, even where they do not take the form of macros or policies.

A.4 Boundary Rule for Future Revisions

Any future revision of the QC specification family should satisfy the following test:

> If a construct cannot be justified as algorithm-agnostic and generically reusable for QC structural description, it should remain in PMS-QC_EXT or in paper prose rather than being promoted into PMS-QC base.

This appendix therefore serves as a drift-control instrument for both editorial revision and schema maintenance.

Appendix B — Canonical Macro Inventory for PMS-QC Base

This appendix provides a compact reference for the **generic PMS-QC base macros**. It is intended as a stable bridge between the main paper and `pms-qc.yaml`. The inventory records macro names, parameters, admissible values, canonical Δ–Ψ expansion order, and the main non-guarantee notes attached to each macro.

These macros are **structural and documentation-oriented objects**. They do not by themselves guarantee physical correctness, convergence, stability, optimal compilation, or deployment adequacy.

B.1 Base Macro Inventory Table

Macro	Parameters	Allowed values	Canonical expansion	Structural role	Non-guarantee note
`QFRAME(kind)`	`kind`	`init`, `problem`, `eigenproblem`, `error_space`	□	Establish or select structural scope	Does not guarantee hardware compatibility, resource availability, or physical adequacy
`QPREP(mode)`	`mode`	`superposition`, `register_init`, `entangle`	A ∘ ∇ ∘ Δ	Structural preparation of registers and amplitude-distribution conditions	Does not guarantee correctness of preparation or physical adequacy
`QORACLE_CALL(oracle_id)`	`oracle_id`	Identifier	Ω ∘ Δ ∘ □	Asymmetry-bearing oracle or controlled-unitary action inside a frame	Does not imply any reusable ladder refinement or algorithm-specific semantics
`QDIFFUSION`	none	n/a	Σ ∘ A	Attractor-driven amplification with structural commit	Does not guarantee convergence, correctness, stability, or physical adequacy
`QFT_ALIGN(direction)`	`direction`	`forward`, `inverse`	A ∘ Φ	Fourier-domain reframing plus alignment	Does not guarantee correctness, stability, or physical adequacy; Σ is not implicit
`QITERATE(k, block)`	`k`, `block`	integer, block	`Θ^k(block)`	Θ-structured repetition of a compositional block	Does not guarantee convergence, termination, correctness, stability, or physical adequacy

Macro	Parameters	Allowed values	Canonical expansion	Structural role	Non-guarantee note
`QMEASURE(basis, register)`	`basis`, `register`	`computational`, `fourier`, or identifier; register identifier	$\Lambda \circ \Delta$	Structural outcome resolution	Does not provide a complete physical theory of measurement
`QSTABILIZE(code_id)`	`code_id`	Identifier	Ψ	Generic structural location for invariants, stabilisation, or code-space maintenance	Does not itself guarantee stability, correctness, or physical adequacy

B.2 Macro-by-Macro Notes

`QFRAME(kind)`

`QFRAME` is the base macro for structural framing. It establishes the scope within which subsequent QC operations are rendered.

Canonical reading:

`QFRAME(kind)` $\Longrightarrow \Box(S)$.

Typical base kinds are:

- `init`
- `problem`
- `eigenproblem`
- `error_space`

Nested or repeated framing is permitted at the structural level, but the macro itself only marks scope. It does not certify that an appropriate physical or computational substrate actually exists.

`QPREP(mode)`

`QPREP` is the base macro for structural preparation. It captures the fact that preparation generally requires a differentiable space, activation, and a resulting stable pattern.

Canonical reading:

`QPREP(mode)` $\Longrightarrow A(\nabla(\Delta(S)))$.

Typical modes are:

- `superposition`
- `register_init`
- `entangle`

The macro is deliberately broad. It provides a structural abstraction of preparation, not an exhaustive taxonomy of preparation procedures.

`QORACLE_CALL(oracle_id)`

`QORACLE_CALL` is the generic base macro for asymmetry-bearing oracle or controlled-unitary action.

Canonical reading:

`QORACLE_CALL(F)` $\Longrightarrow \Omega(\Delta(\Box(S)))$.

Its base semantics are intentionally broad enough to cover:

- Grover-style oracle marking
- controlled-unitary action in QPE-like contexts

But the macro does **not** include any reusable ladder structure. In particular, the explicit power-of-two ladder `U^(2^j)` remains outside base semantics and belongs only to EXT as a paper-specific refinement.

`QDIFFUSION`

`QDIFFUSION` is the base macro for attractor-driven amplification together with integration.

Canonical reading:

`QDIFFUSION` $\Longrightarrow \Sigma(A(S))$.

It is intended for generic Grover-style diffusion or related attractor moves at the structural level. It does not imply:

- fixed-point convergence
- overshoot prevention
- optimal amplitude schedule
- physical adequacy

Those belong to additional refinements, not to the base macro itself.

`QFT_ALIGN(direction)`

`QFT_ALIGN` is the base macro for Fourier-domain reframing and alignment.

Canonical reading:

`QFT_ALIGN(direction)` $\Longrightarrow A(\Phi(S))$.

Important semantic note:

- **Φ acts first**
- then **A**
- an explicit **Σ** may follow if the result is committed

Thus, if a workflow requires commit before measurement, the committed semantic order is:

$\Sigma(A(\Phi(S)))$.

But Σ is not internal to `QFT_ALIGN`; it must be represented explicitly where needed.

`QITERATE(k, block)`

`QITERATE` is the generic base macro for temporally structured repetition.

Canonical reading:

`QITERATE(k, B)` $\Longrightarrow \Theta^k(B(S))$.

The macro does not inspect the internal logic of `block`; it only marks structured repetition. It should therefore not be read as a guarantee of:

- convergence
- termination
- calibration
- correctness

Any stricter interpretation requires additional constraints or refinements.

`QMEASURE(basis, register)`

`QMEASURE` is the base macro for structural outcome resolution.

Canonical reading:

`QMEASURE(basis, register)` $\Longrightarrow \Lambda(\Delta(S))$.

It records that measurement resolves alternatives while leaving unrealised branches structurally marked as non-events. The macro assumes that the relevant measurement frame is available, but does not substitute for physical measurement theory.

`QSTABILIZE(code_id)`

`QSTABILIZE` is the base macro for structural invariance or stabilisation.

Canonical reading:

`QSTABILIZE(code)` $\Longrightarrow \Psi(S)$.

It serves as the generic structural location for:

- stabiliser-code enforcement
- coherence guards
- bounded invariants
- policy-layer constraints in bounded QC settings

But it does not, by itself, prove that the resulting workflow is stable, correct, safe, or implementable.

B.3 Optional vs. Required Σ / Ψ Placements in Base

To avoid drift between prose and schema, the base-layer macro inventory should be read with the following placement discipline:

- `QDIFFUSION` **contains Σ internally**

- `QFT_ALIGN` **does not contain Σ internally**
- `QFT_ALIGN` **may be followed by explicit Σ**
- `QSTABILIZE` is the **explicit Ψ macro**
- standard Grover base rendering contains **no required Ψ**
- QPE base rendering may include **explicit Σ after `QFT_ALIGN(inverse)`**
- reusable ladder-specific and stronger Ψ-specific structures remain **EXT-only**

B.4 Canonical Base Principle

The base macro inventory is intentionally restrained.

Its purpose is not to encode every structurally interesting QC pattern, but to provide a **generic, reusable, algorithm-agnostic macro layer** that remains stable even when the paper adds more specialised reconstructions. Any construct that depends on paper-specific algorithm refinement, stronger convergence claims, or specialised schedule logic should remain outside this appendix and outside the PMS-QC base overlay.

Appendix C — Paper-Specific Refinements Reserved for PMS-QC_EXT

This appendix isolates the constructions that belong to **PMS-QC_EXT** rather than to the generic PMS-QC base layer. Its purpose is to make the extension boundary explicit and to prevent the paper's more specialised reconstructions from being silently reabsorbed into base semantics.

Nothing in this appendix introduces new PMS meta-axioms. All constructs listed here are **paper-specific macro refinements, chain templates, or structural policy refinements** built on top of the existing PMS and PMS-QC layers.

C.1 Refinement Principle

A construct belongs in PMS-QC_EXT when one or more of the following applies:

- it depends directly on a specific algorithmic reconstruction in this paper
- it is not clearly reusable as a generic QC macro
- it introduces stronger stabilisation or schedule logic than the base overlay permits
- it would inflate the base layer with algorithm-specific semantics
- it risks being misread as a validated governance or deployment architecture if left unbounded

This is why the extension layer must remain optional, explicitly subordinate, and clearly marked as paper-derived.

C.2 Reserved EXT Constructs

C.2.1 QPE ladder refinement

Reserved for EXT: explicit reusable macroisation of the controlled power-of-two ladder

U^{2^j}

in QPE-style constructions.

Reason:

- the base layer only needs the generic asymmetry macro `QORACLE_CALL`
- the paper treats the ladder as a specialised refinement
- promoting it to base would silently inflate the generic QC overlay with algorithm-specific structure

Canonical EXT form:

- `QPE_ORACLE_LADDER`
- chain pattern: `0^t(Ω ∘ □)`

This refinement is permitted in EXT precisely because the paper identifies it as methodologically useful while explicitly refusing to treat it as a generic PMS-QC base rule.

C.2.2 Fixed-point and overshoot-regulated Grover refinements

Reserved for EXT: any Grover-style refinement that adds Ψ-bound regulation to standard

amplitude amplification.

Examples:

- fixed-point Grover
- overshoot-regulated centering
- ceiling or basin-entry guards
- attractor schedules designed to restrain amplification dynamics

Reason:

- standard Grover is already fully expressible in the base layer
- the paper explicitly states that standard Grover contains no required Ψ
- adding these refinements to base would wrongly imply that Ψ belongs to the generic Grover construction

Canonical EXT forms include:

- `GROVER_CENTERING`
- `grover_fixed_point`
- `grover_no_overshoot`

These should always be introduced as optional refinements, never as default Grover semantics.

C.2.3 Paper-specific attractor schedules

Reserved for EXT: named attractor schedules or fixed-point-oriented convergence templates.

Examples:

- `ATTRACTOR_FIXED_POINT`
- linear, exponential, or custom attractor schedule templates
- bounded contraction schedules
- algorithm-specific attractor cycling patterns

Reason:

- the base layer includes generic attractor logic via A and base macros such as `QDIFFUSION` and `QFT_ALIGN`
- paper-specific schedules go beyond generic description and begin to encode specialised control structure
- they must therefore remain optional and clearly marked as non-guarantee refinements

C.2.4 Measurement-domain modelling refinements

Reserved for EXT: more elaborate measurement-branch modelling beyond generic `QMEASURE`.

Example:

- `MEASUREMENT_PROJECTION_MODEL`

Reason:

- base `QMEASURE` already captures structural outcome resolution as $\Lambda \circ \Delta$

- additional domain modelling may be useful for paper-aligned analysis, but should not be mistaken for generic base semantics or for a physical theory of measurement

C.2.5 Algorithm-specific stabilisation and policy refinements

Reserved for EXT: policies that go beyond generic base constraints and target specific algorithm families or paper-specific stabilisation concerns.

Examples:

- `qpe_depth_guard`
- `grover_no_overshoot`
- `basis_reframe_guard`
- `attractor_cycle_integrity`

Reason:

- base policies are intentionally generic
- these refinements arise from specific paper arguments about QPE sensitivity, Grover overshoot, or specialised attractor discipline
- they should therefore be treated as **paper-specific structural guardrails**, not as universally preferable defaults

C.3 EXT Chain Templates

The following chain templates belong to EXT because they directly encode the paper's specialised reconstructions rather than generic macro semantics.

`qpe_exact`

A full QPE template with explicit ladder refinement and committed reframing/alignment sequence.

Characteristic feature:

$\Theta^t(\Omega \circ \square)$

is treated as a reusable refinement rather than merely as an inline placeholder.

It also relies on the semantically ordered QFT-alignment stage:

$\Sigma(A(\Phi(S)))$.

`grover_fixed_point`

A Grover variant with additional Ψ-bound regulation inside the iterated structure.

Characteristic feature:

- Ψ is introduced as part of the regulated attractor cycle
- this exceeds the standard Grover base reconstruction

`attractor_alignment`

A paper-specific template foregrounding reframing-plus-alignment patterns in spectral or Fourier-oriented settings.

Characteristic feature:

- useful for analysis
- not required for generic base semantics
- should not be confused with the generic definition of `QFT_ALIGN`

C.4 EXT Guard Language

All EXT constructs should be read under the following guard conditions.

Refinement, not operator addition

EXT does not add new PMS meta-axioms. It introduces only:

- paper-specific macro refinements
- paper-specific chain templates
- optional structural constraints
- specialized macro-level constructs

Optional, not canonical

No EXT construct is required in order to use PMS-QC. A user may work entirely with PMS and PMS-QC base without loading EXT.

Structural, not physical

No EXT refinement proves:

- correctness
- convergence
- physical adequacy
- backend compatibility
- deployment safety

The extension layer only provides additional structural expressiveness for the paper's specialised analyses.

Subordinate, not overriding

EXT must remain subordinate to:

- `PMS.yaml`
- `pms-qc.yaml`

It may refine base macros when explicitly marked as a refinement, but must never silently alter the semantics of the base overlay.

C.5 EXT Reservation Table

Construct	Status	Why EXT-only
`QPE_ORACLE_LADDER`	EXT-only macro refinement	Reusable ladder pattern is paper-specific, not generic base semantics
`GROVER_CENTERING`	EXT-only macro refinement	Fixed-point / overshoot-regulated diffusion is not part of standard Grover base rendering
`ATTRACTOR_FIXED_POINT`	EXT-only refinement template	Specialized convergence schedule logic exceeds base generality
`MEASUREMENT_PROJECTION_MODEL`	EXT-only refinement	More elaborate measurement-branch modelling than generic base requires
`qpe_exact`	EXT-only chain template	Encodes explicit ladder refinement and specialised QPE structure
`grover_fixed_point`	EXT-only chain template	Introduces Ψ-regulated Grover variant beyond base
`attractor_alignment`	EXT-only chain template	Paper-specific analysis template, not generic macro semantics
`qpe_depth_guard`	EXT-only policy refinement	Algorithm-specific depth guard
`grover_no_overshoot`	EXT-only policy refinement	Algorithm-specific overshoot guard
`basis_reframe_guard`	EXT-only policy refinement	Stronger paper-specific reframing discipline
`attractor_cycle_integrity`	EXT-only policy refinement	Specialized attractor-schedule admissibility rule

C.6 Final Boundary Note

The existence of a valuable refinement is not, by itself, a reason to promote it into the base layer.

This is especially important for the present paper because its strongest specialised constructions — QPE ladder refinement, fixed-point Grover regulation, attractor scheduling, and stronger governance heuristics — are precisely the places where an attractive paper-level innovation could otherwise be mistaken for a generic PMS-QC standard.

This appendix therefore serves a conservative methodological function: it keeps the base reusable, the extension explicit, and the paper’s strongest refinements visible without silently transforming them into canon.

Appendix D — Macro Grammar and EBNF Consistency Notes

This appendix records the normalized macro grammar used in the paper and notes the editorial consistency conditions required for synchronization with `pms-qc.yaml` and `pms-qc-ext.yaml`. Its purpose is not to redefine the macro language, but to ensure that the paper, the base overlay, and the extension layer remain aligned in names, parameters, admissible values, expansion order, and the handling of explicit versus optional Σ/Ψ placements.

Nothing in this appendix alters PMS, PMS-QC, or PMS-QC_EXT semantics. It is a redactional and specification-hygiene instrument.

D.1 Normalized EBNF

The grammar below is the normalized form of the macro grammar presented in the main text, adjusted to match the current base YAML discipline in which an explicit `Σ` statement may appear where a workflow must mark a structural commit after a macro that does not contain Σ internally.

```
Program        ::= FrameDecl PrepDecl Block

                   FrameDecl      ::= "QFRAME" "(" FrameKind ")"
                   FrameKind      ::= "init" | "problem" | "eigenproblem" | "error_space"

                   PrepDecl       ::= "QPREP" "(" PrepKind ")"
                   PrepKind       ::= "superposition" | "register_init" | "entangle"

                   Block          ::= Statement | Statement Block
                   Statement      ::= OracleCall
                                    | Diffusion
                                    | Iteration
                                    | Measurement
                                    | Stabilize
                                    | Integrate

                   OracleCall     ::= "QORACLE_CALL" "(" Identifier ")"
                   Diffusion      ::= "QDIFFUSION"
                   Iteration      ::= "QITERATE" "(" Integer "," Block ")"
                   Measurement    ::= "QMEASURE" "(" Basis "," Register ")"
                   Stabilize      ::= "QSTABILIZE" "(" CodeIdentifier ")"
                   Integrate      ::= "Σ"

                   Basis          ::= "computational" | "fourier" | Identifier
                   Register       ::= Identifier
                   CodeIdentifier ::= Identifier
                   Identifier     ::= Letter { Letter | Digit | "_" | "^" | "{" | "}" }
                   Integer        ::= Digit { Digit }
```

D.2 Normalization Notes

The normalized form above reflects five editorial decisions that should remain stable unless

the base specification is explicitly revised.

First, `QFRAME`, `QPREP`, `QORACLE_CALL`, `QDIFFUSION`, `QFT_ALIGN`, `QITERATE`, `QMEASURE`, and `QSTABILIZE` remain the canonical **generic PMS-QC base macros**.

Second, `Σ` is allowed as an **explicit statement** in the workflow grammar. This is necessary because the paper now clearly distinguishes between macros that contain Σ internally and macros that may be followed by Σ externally.

Third, the grammar remains **minimal and structural**. It is not intended to model gate syntax, backend specifics, or full circuit programming languages.

Fourth, the grammar stays **base-layer only**. Paper-specific reusable constructs such as `QPE_ORACLE_LADDER` remain outside this normalized base EBNF and belong only to PMS-QC_EXT.

Fifth, the grammar is deliberately more austere than the possible YAML representation. The YAML may contain extra metadata for toolchains and consistency management, but such metadata must not be mistaken for canonical grammar semantics.

D.3 Canonical Consistency List

The following list should be treated as the editorial checksum for paper/YAML synchronization.

D.3.1 Macro names

The canonical base macro names are:

- `QFRAME`
- `QPREP`
- `QORACLE_CALL`
- `QDIFFUSION`
- `QFT_ALIGN`
- `QITERATE`
- `QMEASURE`
- `QSTABILIZE`

The following are **not** base macro names and must remain EXT-only if used:

- `QPE_ORACLE_LADDER`
- `GROVER_CENTERING`
- `ATTRACTOR_FIXED_POINT`
- `MEASUREMENT_PROJECTION_MODEL`

D.3.2 Parameter names

The canonical base parameter names are:

- `kind` for `QFRAME`
- `mode` for `QPREP`
- `oracle_id` for `QORACLE_CALL` in YAML, while the paper may show `F` or another identifier at the prose level
- `direction` for `QFT_ALIGN`
- `k` and `block` for `QITERATE`
- `basis` and `register` for `QMEASURE`
- `code_id` for `QSTABILIZE`

For paper/YAML synchronization, the prose may use illustrative identifiers, but the schema-facing names should remain stable.

D.3.3 Allowed values

Canonical allowed values in the base layer are:

For `QFRAME(kind)`:

- `init`
- `problem`
- `eigenproblem`
- `error_space`

For `QPREP(mode)`:

- `superposition`
- `register_init`
- `entangle`

For `QFT_ALIGN(direction)`:

- `forward`
- `inverse`

For `QMEASURE(basis, register)`:

- basis may be `computational`, `fourier`, or a domain-specific identifier
- register is an identifier

These value sets should remain identical across the main text, base YAML, and any examples that claim to be base-consistent.

D.4 Canonical Expansion Order

The expansion order below is the paper/YAML synchronization backbone.

Macro	Canonical expansion
`QFRAME(kind)`	$\square$
`QPREP(mode)`	$A \circ \nabla \circ \Delta$

Macro	Canonical expansion
`QORACLE_CALL(oracle_id)`	`Ω ∘ Δ ∘ □`
`QDIFFUSION`	`Σ ∘ A`
`QFT_ALIGN(direction)`	`A ∘ Φ`
`QITERATE(k, block)`	`Θ^k(block)`
`QMEASURE(basis, register)`	`Λ ∘ Δ`
`QSTABILIZE(code_id)`	`Ψ`

The paper should continue to describe `QFT_ALIGN` semantically as **Φ first, then A**, with the chain notation `A ∘ Φ` preserved as the compositional representation already used in the schema family. Where a committed aligned state is needed, the committed reading is:

Σ(A(Φ(S))).

D.5 Optional vs. Required Σ / Ψ Placements

This is the point most likely to drift if not stated explicitly.

D.5.1 Σ placements

- `QDIFFUSION` contains **Σ internally**
- `QFT_ALIGN` does **not** contain Σ internally
- a workflow may add explicit `Σ` after `QFT_ALIGN(...)`
- the EBNF therefore permits `Integrate ::= "Σ"`

Implication: in QPE-like workflows, the paper may show either the semantic expression

Σ(A(Φ(S)))

or the macro sequence

```
QFT_ALIGN(inverse) Σ
```

Both are acceptable so long as the distinction between internal and explicit Σ is preserved.

D.5.2 Ψ placements

- `QSTABILIZE` is the generic explicit Ψ macro in the base layer
- standard Grover contains **no required Ψ**
- QPE base rendering contains **no required reusable Ψ pattern**
- paper-specific Ψ placements in Grover fixed-point variants or QPE-sensitive refinements remain **EXT-only**
- any claim that a given algorithm “naturally requires” a specific Ψ architecture should be avoided in the base grammar

D.6 Where the Paper Stays Minimal and the YAML Must Be More Precise

The paper and the YAML do not carry identical burdens. The paper may remain intentionally lighter in several places, while the YAML must be more explicit.

Paper may remain minimal in:

- illustrative identifiers in examples
- prose-level algorithm walkthroughs
- whether an example is rendered as symbolic composition or macro sequence
- whether a commit boundary is shown as prose or explicit macro statement
- whether helper metadata for parsing or toolchains is shown at all

YAML must be more precise in:

- canonical macro names
- exact parameter names
- allowed values
- base vs. EXT boundary
- explicit handling of internal vs. external Σ
- explicit non-guarantee notes
- policy identifiers
- refinement identifiers
- helper metadata marked as non-canonical

D.7 Drift Warnings

The following drift patterns should be treated as errors or near-errors.

1. **Silent ladder inflation**
   If the paper's QPE example is read as making `U^(2^j)` a generic base macro, the base/EXT boundary has been lost.
2. **Silent Ψ inflation**
   If standard Grover is presented as if Ψ were structurally required, EXT content has leaked into the base rendering.
3. **Silent Σ collapse**
   If `QFT_ALIGN` is treated as if Σ were internal by default, the base macro semantics have been blurred.
4. **Macro/value drift**
   If the paper uses parameter sets or allowed values not reflected in YAML, editorial consistency has broken.

D.8 Final Note

This appendix should be read as a synchronization checkpoint rather than as a new grammar layer. Its function is to keep the paper strong but disciplined: structurally legible in the main text, technically precise in the schemas, and explicit about what belongs to the reusable base versus what remains paper-specific refinement.

Appendix E — Structural Policies: Heuristics, Not Physical Guarantees

This appendix collects the structural policy types discussed across the paper and the current specification family. Its purpose is to make the policy layer legible without overstating what policy language can do.

The central rule is simple:

> **Structural constraint ≠ physical guarantee.**
> **Policy expressiveness ≠ deployment adequacy.**
> **Auditability ≠ validated governance architecture.**

Policies in PMS-QC and PMS-QC_EXT are therefore best read as **structural heuristics, admissibility rules, and bounded governance-oriented guardrails**. They organize what may count as structurally disciplined composition within this framework; they do not certify that a concrete quantum or hybrid AI–QC system is correct, stable, safe, or ready for deployment.

E.1 Policy Taxonomy Overview

The policy families appearing in the paper and schema stack can be grouped as follows:

- **Frame policies**
  Guard the relation between operations and the frames in which they occur.
- **Asymmetry policies**
  Guard Ω-heavy structures that may become unstable unless further bounded.
- **Integration policies**
  Protect Σ-boundaries from unbounded reinterpretation.
- **Self-binding / iteration policies**
  Bound Θ-depth or require stabilisation density.
- **Algorithm-specific refinement policies**
  Add paper-specific structural guardrails for QPE ladders, Grover overshoot control, or attractor schedule integrity.

The first four families belong to the **generic PMS-QC base overlay**. The fifth belongs to **PMS-QC_EXT**.

E.2 Policy Inventory Table

Policy	Structural role	Non-guarantee status	Layer
`no_unframed_omega`	Prevent Ω from appearing without an active frame	Non-guarantee	Base
`measurement_requires_meas_frame`	Require measurement-stable frame before `QMEASURE`	Non-guarantee	Base
`unstable_asymmetry_guard`	Flag Ω-heavy structures as unstable unless further guarded	Non-guarantee	Base
`no_reframing_past_sigma`	Protect Σ-boundaries from unrestricted Φ-reframing	Non-guarantee	Base
`max_iteration_depth`	Bound Θ-depth under generic structural discipline	Non-guarantee	Base
`stabiliser_frequency`	Require sufficient Ψ density in long Θ-chains	Non-guarantee	Base
`qpe_depth_guard`	Bound QPE-style ladder depth in paper-specific contexts	Non-guarantee	EXT

Policy	Structural role	Non-guarantee status	Layer
`grover_no_overshoot`	Add paper-specific overshoot-avoidance discipline to Grover-style cycles	Non-guarantee	EXT
`basis_reframe_guard`	Add stronger paper-specific guard on Φ after Σ	Non-guarantee	EXT
`attractor_cycle_integrity`	Guard paper-specific attractor schedules against structural divergence	Non-guarantee	EXT

E.3 Base Policy Notes

`no_unframed_omega`

Structural role: ensures that asymmetry-bearing actions do not float free of an explicit frame.

It enforces the paper's claim that oracle or controlled action should occur inside an appropriate structural domain. This is a structural admissibility condition, not a physical proof that the actual implementation is correct.

`measurement_requires_meas_frame`

Structural role: ensures that measurement is not introduced without an appropriate measurement-stable frame.

It is a good example of a policy that is structurally sharp but physically modest: it clarifies composition discipline, yet it does not amount to a theory of hardware measurement conditions.

`unstable_asymmetry_guard`

Structural role: marks Ω-heavy compositions as potentially unstable unless further bounded by A or Ψ.

This policy was deliberately softened in the paper from stronger language such as "inadmissible asymmetry" to the more disciplined formulation:

> **structurally unstable unless further guarded**

That phrasing should remain stable.

`no_reframing_past_sigma`

Structural role: records that Σ functions as a structural commit boundary.

It protects the semantic discipline of the macro language, especially where reframing and integration could otherwise be conflated. Again, it is a structural rule of the representation layer, not a proof about compiled or executed systems.

`max_iteration_depth`

Structural role: places a generic bound on Θ-structured repetition unless additional stabilisation is declared.

This does not prove that a bounded iteration count is the right one physically or algorithmically; it only enforces a disciplined structural ceiling within the overlay.

`stabiliser_frequency`

Structural role: requires periodic stabilisation in long repetitive workflows.

It is a reusable generic policy, but it should still be read as a structural heuristic rather than a validated recipe for fault tolerance or safe orchestration.

E.4 EXT Policy Notes

`qpe_depth_guard`

Structural role: a paper-specific bound on QPE-style ladder depth.

It belongs to EXT because it arises from the paper's refined handling of the controlled-unitary ladder and is not needed for the generic base overlay.

`grover_no_overshoot`

Structural role: adds paper-specific discipline to Grover-style cycles when overshoot-regulated variants are considered.

It must remain EXT-only because standard Grover in the paper is explicitly base-expressible without Ψ-bound overshoot logic.

`basis_reframe_guard`

Structural role: stronger paper-specific guard on Φ after Σ.

It extends the generic base policy `no_reframing_past_sigma` rather than replacing it.

`attractor_cycle_integrity`

Structural role: guards paper-specific attractor schedule structures.

It is useful where EXT introduces stronger schedule semantics, but it should never be misread as a proof that a particular attractor schedule is physically convergent.

E.5 Three Explicit Clarifications

E.5.1 Structural constraint ≠ physical guarantee

A chain can be structurally well-formed under PMS-QC and still be:

- physically noisy
- numerically unstable
- badly compiled
- hardware-inadequate
- scientifically unconvincing for a given task

Structural discipline helps, but it does not replace domain-specific validation.

E.5.2 Policy expressiveness ≠ deployment adequacy

The fact that a workflow can be expressed, bounded, audited, and policy-checked at the Δ–Ψ level does **not** show that it is fit for real deployment.

This is especially important in the governance chapter: a policy-rich representation may be analytically helpful while still falling far short of a validated control architecture.

E.5.3 Auditability ≠ validated governance architecture

PMS-QC can increase structural legibility and auditability. That is already valuable. But auditability alone does not yield:

- operational safety
- accountability sufficiency
- system robustness
- deployment readiness
- governance validation

The paper now correctly treats governance as **conditional architectural option**, not natural culmination. The policy layer must therefore be read in the same way.

E.6 Policy Layer and Base/EXT Discipline

The policy layer also plays a repository-governance role.

- **Base policies** should remain generic and reusable.
- **EXT policies** should remain paper-specific and optional.
- No algorithm-specific guard should silently migrate into the generic base overlay.
- No policy should be phrased as if it were a physical law or validated engineering default.

This is one of the main reasons the policy appendices are useful: they help keep YAML growth from turning paper-specific guardrails into pseudo-canonical base architecture.

E.7 Final Note

The correct methodological stance is neither to dismiss the policy layer nor to inflate it. Structural policies matter because they preserve discipline, legibility, and boundedness. But their value lies precisely in being read for what they are:

- heuristics
- structural constraints
- admissibility guides
- bounded governance-oriented tools

—not as substitutes for physics, proof, calibration, or accountable system validation.

Appendix F — Grover Reconstruction: Base vs. Optional Stabilisation

This appendix restates the Grover reconstruction in a compact form that separates the **generic base rendering** from **optional EXT-only stabilisation refinements**. Its purpose is to make the layer distinction explicit while relieving the main chapter of some boundary work.

The main principle is:

> **Standard Grover belongs to the PMS-QC base layer. Ψ-augmented fixed-point or overshoot-regulated variants belong to PMS-QC_EXT.**

Nothing in this appendix should be read as replacing standard quantum descriptions of Grover's algorithm. It provides only a bounded structural rendering within the PMS-QC framework.

F.1 Standard Grover as Base Chain

The standard Grover workflow in base macro form is:

```
QFRAME(problem)
                        QPREP(superposition)
                        QITERATE(k, { QORACLE_CALL(F); QDIFFUSION; })
                        QMEASURE(basis=computational, register=search)
```

Its structural expansion can be rendered compactly as:

[Λ \circ Δ \circ Θ^k(Σ \circ A \circ Ω \circ Δ \circ □).]

This is the canonical **base-layer Grover chain** in the current PMS-QC rendering.

F.2 Structural Reading of the Base Chain

The base interpretation proceeds as follows:

- □ establishes the problem frame
- **Δ** distributes alternatives through the prepared search space
- **Ω** marks asymmetry through oracle action
- **A** drives attractor-like amplification toward the marked state
- **Σ** commits the diffusion effect at each Grover cycle
- **Θ^k** repeats the oracle–diffusion block
- **Δ / Λ** resolve the outcome structure at measurement

This is sufficient for the paper's bounded claim that standard Grover can be rendered clearly within the base grammar.

F.3 What the Base Chain Does Not Claim

The base rendering of Grover does **not** by itself claim:

- physical correctness
- numerical optimality
- convergence proof
- robustness under noise

- implementation adequacy
- unique explanatory status over standard QC formalisms

It is a structural reconstruction, not a replacement theory.

F.4 Optional Ψ-Augmented Variants as EXT-Only

The paper explicitly reserves stronger stabilisation variants for PMS-QC_EXT. These include:

- fixed-point Grover
- overshoot-regulated Grover
- centering variants
- additional convergence-bounding attractor schedules

A typical EXT-style regulated cycle may be rendered as:

[Θ^k(Ψ \circ Σ \circ A \circ Ω \circ Δ \circ □).]

Or, using an explicit refinement macro such as `GROVER_CENTERING`, as a regulated amplification structure in which Ψ is introduced after attractor integration.

These variants remain **optional** and **paper-specific**.

F.5 Why Ψ Remains Optional in Standard Grover

The paper's position is methodologically important:

- standard Grover does **not** require an explicit Ψ macro in its base rendering
- Ψ enters only when the analysis moves toward fixed-point control, overshoot prevention, or stronger bounded-governance heuristics
- therefore, introducing Ψ into every Grover reading would incorrectly collapse the base/EXT distinction

This is exactly why the current schema family keeps `QDIFFUSION` generic and reserves `GROVER_CENTERING` and related refinements for EXT.

F.6 Compact Base vs. EXT Table

Grover form	Layer	Chain sketch	Status
Standard Grover	Base	`Λ ∘ Δ ∘ Θ^k(Σ ∘ A ∘ Ω ∘ Δ ∘ □)`	Canonical base rendering
Fixed-point Grover	EXT	`Θ^k(Ψ ∘ Σ ∘ A ∘ Ω ∘ Δ ∘ □)` or equivalent refined form	Optional paper-specific refinement
Overshoot-regulated Grover	EXT	Base chain plus explicit Ψ-bound regulation	Optional paper-specific refinement
Centering variant	EXT	`GROVER_CENTERING`-type refinement	Optional paper-specific refinement

F.7 IA-Analogy Guard Note

The main chapter introduces an IA-style analogy in highly guarded form. For clarity, the compact guard is:

> The IA-reference in the Grover discussion is a **structural analogy only**. It does **not** anthropomorphise the algorithm, does **not** attribute genuine awareness or agency to quantum states, and does **not** license an unmarked transfer of anthropological PMS

> derivations into QC theory.

The point of the analogy is only that the oracle-superposition relation can be rendered as a formally asymmetric relation between distributed alternatives and selective action. Nothing stronger should be inferred.

F.8 Non-Guarantee Clause for Grover Reconstruction

The Grover reconstruction in both base and EXT variants should always be read under the following non-guarantee clause:

- structural rendering does not imply physical correctness
- optional stabilisation does not imply proven convergence
- Ψ-augmentation does not imply validated control architecture
- interpretive compactness does not imply exhaustive capture of the algorithm

F.9 Final Note

Grover is a good showcase for the PMS-QC framework precisely because the layer separation can be made cleanly:

- the **base layer** is already strong enough to render the standard algorithm
- the **extension layer** can add paper-specific regulation without contaminating the base

That is the right methodological outcome: not weakening the reconstruction, but disciplining its reach.

Appendix G — QPE Reconstruction: Ladder Refinement Without Base Inflation

This appendix restates the Quantum Phase Estimation (QPE) reconstruction in a way that sharpens the boundary between the **generic PMS-QC base layer** and the **optional PMS-QC_EXT refinement layer**. Its purpose is to stabilize the interpretation already given in Chapter 7 and to prevent the power-of-two ladder from being silently reabsorbed into base semantics.

Nothing in this appendix replaces standard QC descriptions of QPE. It records only a bounded structural rendering within the PMS-QC framework. The paper's own methodological boundary remains decisive: the explicit ladder (U^{2^j}) is treated as a **paper-specific macro refinement**, not as a generic PMS-QC base rule.

G.1 Minimal Base Reading of QPE

The minimal base-level macro rendering is:

```
QFRAME(eigenproblem)
                        QPREP(superposition)
                        QITERATE(t, { QORACLE_CALL(U^{2^j}); })
                        QFT_ALIGN(inverse)
                        Σ
                        QMEASURE(basis=computational)
```

This is admissible as a **base skeleton** because `QORACLE_CALL(U^{2^j})` is used here only as a minimal placeholder for controlled-unitary action, not as a reusable ladder macro. The reusable ladder belongs only to EXT.

A compact structural rendering is:

[Λ \circ Δ \circ Σ \circ A \circ Φ \circ Θ^t(Ω \circ □) \circ A \circ ∇ \circ Δ.]

This expression should be read as a base-compatible structural chain, not as a claim that the entire specialized ladder logic has already been promoted into the generic overlay.

G.2 Structural Reading of the Base Chain

The base reading proceeds in five stages.

First, `QFRAME(eigenproblem)` establishes the eigenproblem frame (□), which scopes the composite control-target structure. Second, `QPREP(superposition)` produces the distributed Δ-field in the control register via the usual preparation chain (A \circ ∇ \circ Δ). Third, `QITERATE(t, { QORACLE_CALL(U^{2^j}); })` renders ordered controlled asymmetry as a Θ-structured sequence of Ω-bearing actions. Fourth, `QFT_ALIGN(inverse)` followed by explicit Σ yields the committed readout-ready configuration. Fifth, `QMEASURE` resolves the resulting alternatives as (Λ \circ Δ).

G.3 Explicit EXT Reservation of the Ladder

The ladder boundary is the central methodological point.

The paper states that the structural rendering

$$\Theta^t(\Omega \circ \square)$$

is fully compatible with the base grammar, while the **named, reusable macro-pattern** for the power-of-two ladder remains paper-specific and therefore EXT-only. For that reason, `QORACLE_CALL` stays the generic base macro, whereas the explicit `U^{2^j}` ladder belongs only to the optional extension layer.

This distinction matters because otherwise the base overlay would be inflated with algorithm-specific semantics. The paper's own rule is clear: useful specialized structure may be formalized, but it must not be silently promoted into the generic base layer.

G.4 The Required Ordering: Φ → A → Σ

The QFT stage must be read in the fixed semantic order:

$$\Phi \to A \to \Sigma$$

In macro form, that is rendered as:

```
QFT_ALIGN(inverse) Σ
```

with the concise Δ–Ψ expression:

$$\Sigma(A(\Phi(S)))$$

This ordering is important for three reasons. It preserves the distinction between reframing and diffusion, it keeps `QFT_ALIGN` separate from `QDIFFUSION`, and it avoids ambiguity about where the structural commit actually occurs. Φ acts first as representational reframing, A then aligns the reframed amplitudes, and Σ finally commits the aligned structure for measurement.

G.5 Possible Ψ Loci Without Architecture Privilege

The paper allows multiple **possible structural loci** for Ψ in QPE, but explicitly refuses to privilege any single one as uniquely natural, optimal, or required.

The possible loci include:

- Ψ constraining the controlled-unitary ladder
- Ψ bounding phase-estimation error after Σ
- Ψ stabilizing the frame during Φ-based reframing

These should be read only as **possible stabilisation locations within the grammar**, not as a validated control architecture or a necessary QPE completion.

G.6 Base-Compatible vs. EXT-Only

QPE element	Status	Layer	Reason
`QFRAME(eigenproblem)`	Base-compatible	PMS-QC base	Generic frame macro
`QPREP(superposition)`	Base-compatible	PMS-QC base	Generic preparation macro
`QORACLE_CALL(U^{2^j})` in a base example	Base-compatible placeholder	PMS-QC base example	Allowed as minimal rendering only
`Θ^t(Ω ∘ □)` as abstract structure	Base-compatible	PMS-QC base	Generic structural rendering
Explicit reusable (U^{2^j}) ladder macro	EXT-only	PMS-QC_EXT	Algorithm-specific refinement
`QPE_ORACLE_LADDER`	EXT-only	PMS-QC_EXT	Named ladder refinement
Stronger QPE-specific Ψ placement rules	EXT-only	PMS-QC_EXT	Paper-specific stabilisation refinement

This is the cleanest way to preserve both strengths at once: a strong base rendering and a disciplined EXT boundary.

G.7 Non-Guarantee Clause

The QPE reconstruction here does **not** establish:

- physical correctness
- precision guarantees
- convergence guarantees
- implementation adequacy
- optimal stabilisation placement
- validated governance architecture

As the paper states, all policies and constraints in this context are structural in nature and do not guarantee physical correctness, convergence, or computational efficiency of concrete implementations.

G.8 Final Note

QPE is a particularly good test case because it shows that PMS-QC can remain both expressive and disciplined. The base layer is strong enough to render the algorithm's structural skeleton, while the extension layer remains the proper place for ladder refinement and stronger stabilisation logic. That is exactly what "ladder refinement without base inflation" is meant to preserve.

Appendix H — Governance Application Boundaries

This appendix gathers the methodological boundary conditions for governance-oriented use of PMS-QC. Its purpose is not to weaken Chapter 8, but to keep its architectural proposals explicitly bounded and fully consistent with the paper's anti-finality and non-overclaim discipline.

The key point is simple: governance-related use of PMS-QC is a **conditional architectural option**, not the necessary endpoint of the framework. One may use PMS-QC for structural description, macro comparison, or paper-specific refinement without entering policy-layer or control-layer applications at all.

H.1 Governance Conditionality

The governance-oriented reading developed in the paper is explicitly conditional. PMS-QC **may** be used to articulate auditability, constraint management, and bounded coordination in hybrid AI–QC settings, but this does not mean that governance is the natural telos of PMS-QC, nor that formal policy expressiveness by itself yields a deployable or normatively sufficient control system.

This means the governance chapter should always be read as exploring one bounded extension path inside the framework, not as establishing the uniquely correct system architecture for future agentic quantum computing.

H.2 Application Boundary Note

The paper's application boundary is explicit and should be repeated in compact form here:

- no clinical interpretation of QC actors or hybrid systems
- no personal interpretation
- no moral interpretation
- no anthropomorphisation of quantum processes
- no silent promises of safety, capability, or deployment readiness

This is not a decorative caution. It is part of the methodological discipline that keeps structural rendering from sliding into person-language, moralization, or unearned system claims.

H.3 Audit Layer Boundary

The paper allows PMS-QC to be used as an operator-level audit mechanism through structural replay, ex post inspection, and structural diagnostics of Δ–Ψ chains. That is a real strength. But the paper also states that such an audit layer does **not** by itself amount to a validated safety system, comprehensive audit guarantee, or deployment-ready governance framework.

So the proper conclusion is:

- auditability is valuable
- structural replay is useful
- diagnostics improve legibility

but none of these by themselves validate a real-world governance architecture.

H.4 Policy-Layer Boundary

The policy language in Chapter 8 is intentionally labeled as **structural governance heuristics and constraints** rather than as a self-sufficient normative safety architecture. The listed policy classes—frame, integration, self-binding, and reframing policies—may be instantiated in the specification and may support auditing, compilation, and agentic control, but policy expressiveness must not be confused with deployment adequacy, verified safety, or sufficient runtime control in real systems.

This gives the policy layer a strong but bounded role:

- it can constrain admissible structures
- it can improve machine-checkable legibility
- it can support bounded governance reasoning

but it does not validate the underlying architecture.

H.5 Multi-Frame Boundary

The paper allows a hybrid system to be rendered through interacting physical, computational, business/task, and governance frames. Φ, X, Σ, and Ψ are used to describe transitions, isolations, commitments, and persistent constraints across these domains. But the paper immediately adds the humility clause: a formally well-described multi-frame system is not, for that reason alone, a safe or validated one.

That is the right boundary formula for this appendix:

> multi-frame structural description ≠ secure agent architecture

H.6 LLM / Agent Note

The LLM section is now deliberately cautious. PMS may serve as a **possible structuring interface for bounded experimental architectures**, not as a fully established prompt- or inference-governance solution. Prompting and inference may be rendered through Δ–Ψ structures, but this does not show that PMS already yields a robust, sufficient, or deployment-ready orchestration architecture for LLM-based QC systems.

So the correct bounded reading is:

- PMS-QC may structure the agent-facing layer
- PMS-QC may support bounded experimental orchestration
- PMS-QC does not thereby become a validated LLM governance solution

H.7 Meta-Compiler Humility Clause

The meta-compiler proposal is useful, but the paper gives it a strict humility clause. PMS may sit above QC frameworks as an architectural separation layer in which the backend handles physical correctness, PMS handles structural correctness, and the agent handles goal-directed reasoning. But this division of labour is presented only as an **architectural separation proposal**, not as a validated deployment template. A PMS-described meta-compiler does not

by itself establish safe execution, reliable orchestration, or production-worthy control.

This should remain one of the clearest boundary statements in the whole governance apparatus:

> meta-compiler layer ≠ validated control system

H.8 Relation to PMS Application Discipline

At the repository level, this appendix should also be read as aligned with the broader PMS application discipline: boundedness, reversibility, non-finality, and non-anthropomorphic handling of structurally loaded claims. In the QC paper, this shows up as governance conditionality, application-boundary language, and repeated refusal to convert structural description into safety or deployment claims. The point is not to copy the PMS base application gate verbatim, but to mirror its discipline in a QC-appropriate form.

H.9 Final Boundary Formula

The shortest correct summary of this appendix is:

- governance use is conditional
- policy language is heuristic and structural
- auditability is not validation
- LLM-facing structure is experimental, not established
- meta-compiler separation is a proposal, not a deployment template
- no clinical, personal, moral, or anthropomorphising transfer is licensed

That is the governance discipline the paper now consistently maintains.

Appendix I — Terminology and Boundary Glossary

This appendix provides a compact glossary of the key boundary terms that now carry the paper's methodical discipline. Its purpose is to reduce drift across the paper, the examples, and the schema family. Several entries are already aligned with `pms-qc.yaml`, especially where the schema formalizes glossary-level distinctions such as `generic_base_macro`, `paper_specific_refinement`, `optional_extension`, `non_guarantee`, `structural_comparability`, and `structural_not_physical`.

I.1 Portable

Portable means usable across more than one domain at the level of structural description within the present framework. It does **not** mean universally final, exhaustively adequate, or equally explanatory in every context.

In this paper, portability is methodological strength, not final interpretive authority. That is why the paper consistently speaks of bounded cross-domain comparison rather than universal closure.

I.2 Cross-domain

Cross-domain means that the same operator vocabulary may be used to render structures in different domains—praxeological, computational, agentic, or quantum-computational—without implying that these domains are physically, ontologically, or explanatorily identical.

Cross-domain language should therefore always be read together with the paper's non-identity and non-monopoly discipline.

I.3 Structural rendering

A **structural rendering** is a description of a phenomenon through the Δ–Ψ grammar at the level of composition, framing, asymmetry, iteration, integration, and related structural roles.

A structural rendering is not a claim that the rendered phenomenon is "really" made of PMS operators. It is an operator-level way of making organisation legible within this framework.

I.4 Structural comparability

Structural comparability is a relation of operator-level comparability across domains. In the schema family, it is defined as a relation that does **not** imply ontological identity or equal explanatory depth.

This term is especially useful when the paper says that QC, software, and praxeological systems may be rendered through the same vocabulary while still differing in local semantics, explanatory depth, and physical basis.

I.5 Non-identity

Non-identity means that a mapping from PMS operators to QC roles is not an ontological identity claim. Quantum phenomena are not "really" PMS operators; rather, they can be rendered through the PMS-QC overlay at a structural level. The same discipline applies

wherever the paper compares domains through shared operator roles.

I.6 Paper-specific refinement

A **paper-specific refinement** is a construct, chain pattern, or policy refinement that depends directly on a particular reconstruction, schedule logic, stabilisation rule, or governance discussion developed in the present paper and therefore does not belong to the reusable generic base layer. `pms-qc.yaml` defines it precisely in this sense.

Typical examples are:

- the explicit reusable QPE ladder
- fixed-point or overshoot-regulated Grover variants
- stronger attractor schedules
- algorithm-specific stabilisation heuristics

I.7 Generic base macro

A **generic base macro** is a macro belonging to the reusable PMS-QC base overlay and intended for algorithm-agnostic structural use. In the schema family, examples include `QFRAME`, `QPREP`, `QORACLE_CALL`, `QITERATE`, `QDIFFUSION`, `QFT_ALIGN`, `QMEASURE`, and `QSTABILIZE`.

A generic base macro may be broad, but that breadth must not be used to smuggle paper-specific refinements into base semantics.

I.8 Optional extension

An **optional extension** is a non-canonical add-on layer that refines or augments a bounded use case while remaining subordinate to PMS and the PMS-QC base overlay. That is the exact sense in which PMS-QC_EXT is to be understood.

“Optional” here means more than merely technically loadable. It also means:

- not required for generic base description
- not canonical PMS
- not silently promotable into base
- not entitled to redefine PMS or PMS-QC semantics

I.9 Non-guarantee

A **non-guarantee** is an explicit indication that structural admissibility does not imply physical correctness, safety, convergence, or deployment adequacy. This is one of the most important glossary terms because it protects the whole paper/schema family from overclaim.

Wherever a macro, policy, or refinement is marked by a non-guarantee discipline, the correct reading is:

- structurally meaningful
- potentially useful
- not physically certified
- not operationally validated merely by being well-described

I.10 Structural, not physical

Structural, not physical means that the relevant claim concerns rendering, organisation, annotation, or admissible composition, not the underlying physical mechanism itself. The schema family formalizes this distinction explicitly, and the paper repeats it in its treatment of mappings, macros, policies, and governance proposals.

This term is especially useful when distinguishing:

- operator rendering from hardware behavior
- structural policy from safety guarantee
- macro abstraction from compilation correctness
- auditability from governance validation

I.11 Governance conditionality

Governance conditionality means that governance- or policy-layer use is one bounded architectural option within the framework, not the necessary culmination of PMS-QC and not a validated enforcement architecture. This phrase should anchor every reading of Chapter 8.

I.12 Boundary formula

The glossary can be compressed into one boundary formula that is worth reusing across paper, examples, and YAML:

> portable does not mean final cross-domain does not mean identical structural does not mean physical policy-rich does not mean deployment-ready refined does not mean canonical

That is the drift-prevention logic now running through the full PMS-QC stack.